
Do Large Language Models Detect and Exploit Decisive Sequential Information?

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Sequential prediction mixes event identity with temporal order, making it difficult
2 to tell whether models exploit genuine order-sensitive structure or local statistical
3 shortcuts. We study this question with a synthetic benchmark over letter sequences
4 that removes semantics while varying hidden key subsets, ordering complexity,
5 and lag. For compliance tasks defined by hidden ordered key subsequences, we
6 prove that under independent full-support sampling local information leakage is
7 unavoidable, so this family cannot be purely sequential even in the single-position
8 mutual-information sense. We also analyze a hidden-subset parity family and show
9 that, in the balanced case, it is k -locally pure for every window shorter than the
10 full sequence, motivating a stricter global-integration test. We therefore compare
11 models against k -gram baselines and theoretical AUC ceilings under label noise.
12 Across autoregressive LLMs, encoder-based transformers, and recurrent models,
13 we find that: (i) both large decoders and small encoders can approach the optimum
14 in deterministic compliance settings, but encoders are orders of magnitude more
15 parameter- and data-efficient; (ii) under label noise, decoders up to 8B remain near
16 k -gram level while encoders stay close to the optimum; and (iii) in our hidden-
17 subset parity regime, where the tested k -gram baselines remain at chance, all tested
18 models stay at chance under standard supervised training. The benchmark thus
19 serves as a diagnostic for order-sensitive prediction beyond local shortcuts.

20 1 Introduction

21 Predicting from event histories can depend on more than which events occurred; it can also depend
22 on when and in what order they occurred. In longitudinal health data, the same diagnoses can imply
23 different risk depending on their temporal trajectory, and in recommendation the same consumed
24 items can reflect different user states when encountered in different orders. This paper studies exactly
25 this regime: tasks in which temporal order contributes predictive signal beyond event identity alone.

26 This regime is not merely theoretical. Recent work on longitudinal EHRs suggests that ordered
27 trajectories can add predictive value beyond unordered summaries [Edelman et al., 2026, Beaney
28 et al., 2024, Placido et al., 2023, Pollington et al., 2026]. For example, in primary-care EHRs
29 from 6.29 million patients, Beaney et al. [2024] report that sequence-based disease representations
30 consistently outperform bag-of-words alternatives, though the absolute gains are modest. The gains
31 are heterogeneous across tasks, but they establish that clinically relevant order-sensitive settings exist.

32 At the same time, large language models (LLMs) and related transformer-based sequence models
33 are increasingly used in domains where such order-sensitive signal may matter, including clinical
34 event prediction [Savcisen et al., 2023, Shmatko et al., 2025, Li et al., 2020, Rasmy et al., 2021],
35 sequential recommendation [Kim et al., 2025], and temporal reasoning [Fatemi et al., 2024]. The
36 key question is therefore not whether order ever matters, but whether current architectures actually

learn to use it when it is present. Strong performance on sequential tasks is often taken as evidence of sequential understanding, yet in principle the same performance could arise from unordered content or from local statistics.

Recent evidence motivates that concern. Shuffling item order causes only small degradation in LLM-based recommendation [Kim et al., 2025]; clinical analyses report reliance on statistical pattern matching rather than genuine temporal reasoning [Bedi et al., 2025]; and transformer predictions can often be approximated by simple k -gram rules [Nguyen, 2024]. Together, these findings suggest that apparent sequential competence may often arise from local statistical shortcuts rather than global sequence integration [McCoy et al., 2019, Geirhos et al., 2020, Mirzadeh et al., 2024].

Testing this on real data is difficult because semantic content and temporal structure are entangled. A diagnosis can be predictive because of what it is, because of where it appears in a trajectory, or because of both. Strong performance on real-world sequential tasks therefore does not reveal what signal the model used. Moreover, many benchmarks that aim to evaluate sequential ability do not actually require much sequential understanding [Klenitskiy et al., 2024]. We address this by constructing a controlled benchmark based on letter sequences, where semantic reasoning is effectively out of scope. Random letter sequences do not form meaningful linguistic units, and the task bears no relation to natural-language semantics. This removes confounders such as world knowledge and content-based inference, allowing us to isolate what models learn from order-sensitive structure itself. Our aim is therefore not to evaluate instruction following or in-context reasoning, but to test what these architectures can learn from labeled order-sensitive data once semantic cues are removed.

Our theoretical analysis then identifies an important limitation of the most natural compliance-style ordering tasks: under independent sampling with full support, they inevitably leak local information. Thus this benchmark family cannot be made completely free of local cues, because order constraints induce statistical regularities that remain visible from limited views of the sequence. We formalize this as an impossibility result in the single-position mutual-information sense $I(Y; X_t) = 0$ (Theorem 3.3). This result is specific to the compliance family and does not extend to arbitrary order-sensitive labels. It therefore shifts the question from a binary one, whether models use sequential structure at all, to a quantitative one: how much of a task is recoverable from local patterns such as k -grams, and when is genuine global integration required?

To study that question, we compare autoregressive LLMs, encoder-based transformers, and recurrent architectures against k -gram baselines and theoretical performance bounds. Three findings emerge:

1. **Scale and architecture matter.** In deterministic compliance settings, both sufficiently large decoder-only models and small encoder-based architectures can approach the theoretical optimum, but encoders do so with orders of magnitude fewer parameters and less data.
2. **Noise raises the scale requirements.** Under label noise, decoder-only models that succeed in the deterministic regime remain near the k -gram level up to 8B parameters, whereas encoders stay near the theoretical ceiling.
3. **Hidden-subset parity stays hard.** On a hidden-subset parity task designed to stress global integration, and on which the tested k -gram baselines remain at chance, none of the tested models exceeds chance under standard supervised training.

We view our benchmark as a diagnostic tool. It isolates the order-sensitive component that realistic domains such as clinical trajectories and recommendation mix with semantics, without claiming to reproduce their full richness. In that sense, it is designed to ask what models are exploiting when they appear to succeed on sequential prediction problems, rather than to serve as a substitute for real-world data.

Contributions.

- We introduce a synthetic benchmark with tunable parameters that separates order-sensitive structure from semantic content, enabling controlled evaluation of whether models exploit sequential signal beyond local shortcuts.
- For compliance tasks, we prove that under independent sampling with full support local information leakage is unavoidable in the single-position mutual-information sense, and under the uniform proposal law we derive an explicit lower bound on that leakage.

- For the balanced parity family, we prove k -local purity for every window shorter than the full sequence, giving a complementary construction in which bounded local views carry no signal by design.
- We derive theoretical performance bounds under label noise and empirically show a sharp architecture-efficiency gap: encoder-based models reach theoretical limits with far fewer parameters than decoder-only models, while the hidden-subset parity regime remains unsolved by all tested models under standard supervised training.

2 Related Work

A growing body of work suggests that temporal order can add predictive value beyond unordered event identity in EHRs. Edelman et al. [2026] identify ordered clinical trajectories that provide interpretable signal beyond unordered features for adverse-outcome risk estimation. Beaney et al. [2024] show that sequence-based disease representations outperform bag-of-words alternatives in primary-care EHRs, and Placido et al. [2023] report stronger pancreatic-cancer risk prediction from time-sequential disease trajectories than from non-sequential baselines. Pollington et al. [2026] summarize this broader literature and report moderate but heterogeneous gains from temporal over static models. These settings motivate our question but cannot answer it directly, because they are semantically rich and do not isolate whether the gains come from temporal order, event semantics, or their interaction. Transformer-based models are now routinely applied to longitudinal health and behavior data. In healthcare, BEHRT, Med-BERT, life-event models, and Delphi-style architectures all use event histories to predict downstream outcomes [Li et al., 2020, Rasmy et al., 2021, Savcisen et al., 2023, Shmatko et al., 2025]. In recommendation, LLM-based systems are increasingly used to model user sequences [Kim et al., 2025]. These works establish that sequence models can be useful in practice, but they are typically evaluated in semantically rich environments where content and order are intertwined.

Several recent benchmarks directly probe temporal or sequential ability. Test of Time controls temporal structure in synthetic graphs and shows that performance declines as temporal complexity increases [Fatemi et al., 2024]. TIMER-Bench studies longitudinal clinical timelines and reports a lost-in-the-middle effect [Cui et al., 2025]. Klenitskiy et al. [2024] argue that many evaluation datasets marketed as sequential do not in fact require strong sequential modeling, which complicates the interpretation of strong performance on such benchmarks. Complementary analyses suggest that transformer behavior is often explainable by local statistics rather than global sequence computation. Nguyen [2024] show that transformer predictions can frequently be approximated by k -gram rules, and similar patterns have been reported in recommendation and clinical reasoning [Kim et al., 2025, Bedi et al., 2025]. These findings connect to the broader shortcut-learning literature [McCoy et al., 2019, Geirhos et al., 2020, Mirzadeh et al., 2024]. At the same time, induction-head-style associative recall shows that transformers can realize dependencies beyond fixed k -gram rules [Olsson et al., 2022, Gu and Dao, 2024], so the relevant question is not whether such mechanisms exist in principle, but when they are learned on a given task family. To this end, synthetic benchmarks are useful when the goal is not only to measure performance but to identify the signal a model is using. Recent work has introduced tunable synthetic benchmarks for reasoning and sequence processing [Kaiser et al., 2025, Ramezanali et al., 2025]. Our benchmark differs in that it is designed to strip away semantics while supporting theoretical analysis of the remaining information structure, including unavoidable local leakage in compliance tasks and a parity-based family that stresses global integration.

3 Methods and theory

In this section, we formalize the notion of sequential dependence, prove fundamental limits on what such tasks can test, and describe our experimental framework.

Illustrative example. Consider predicting adverse cardiac outcomes from a patient’s medical history. A clinician knows that certain diagnostic events in a specific order signal elevated risk: for instance, *diabetes* followed by *hypertension* followed by *atrial fibrillation*; with sufficient time between events, they may indicate progressive cardiovascular deterioration. Other diagnoses (respiratory infections, minor injuries) appear in the record but are not predictive. The challenge is that neither

the relevant diagnoses, their critical ordering, nor the required temporal spacing are known a priori, they must be learned from data.

This structure, a subset of events in a specific order with temporal lag constraints, embedded among irrelevant events, arises across domains: clinical trajectories, user behavior sequences, financial transactions, and more. We abstract it into a controlled framework using letter sequences, where each event type maps to a letter. This abstraction intentionally strips away semantic content: “diabetes” becomes simply D, losing its medical meaning. This is by design, we aim to isolate and study the purely sequential nature of these phenomena, separate from domain knowledge. Consequently, we do not expect models to leverage prior knowledge about the events themselves, which motivates our fine-tuning approach. We also randomize the ordering rule κ to avoid coincidental alignment with alphabetical order (except in our “naive” baseline setting), ensuring that success requires learning the hidden structure rather than exploiting pre-existing associations.

In our clinical example: diabetes $\rightarrow$ D, hypertension $\rightarrow$ H, atrial fibrillation $\rightarrow$ A, with non-predictive diagnoses mapping to other letters. A patient’s history becomes a sequence like BXDCAQZHELMPTKDWSANJ (length $n = 20$, alphabet size $\ell = 26$). The *key subset* $\mathcal{S} = \{D, H, A\}$ contains the predictive letters (size $m = 3$), with a ordering rule κ : $\kappa(D) < \kappa(H) < \kappa(A)$. The *lag* parameter λ captures temporal spacing, with $\lambda = 3$, we examine positions $(1, 4, 7, \dots)$, then $(2, 5, 8, \dots)$, etc. More generally, λ can be a vector $(\lambda_1, \lambda_2, \dots)$ to model phenomena where temporal constraints evolve, for instance, requiring events to occur progressively closer together as a condition worsens. We say that a sequence is *compliant* if key letters appear in the prescribed order at positions separated by the specified lag(s). Hence, the model must discover: (i) which letters matter, (ii) the decisive ordering, and (iii) the lag structure, none of which are provided explicitly.

3.1 Setup and Notation

Let $\mathcal{A}$ be an alphabet of size $\ell = |\mathcal{A}|$. To instantiate one task, we first sample a hidden key subset $\mathcal{S} \subseteq \mathcal{A}$ of size $m = |\mathcal{S}|$ uniformly among all m -subsets of $\mathcal{A}$, and then sample a bijection $\kappa : \mathcal{S} \rightarrow \{1, \dots, m\}$ uniformly among all such bijections. The realized pair $(\mathcal{S}, \kappa)$ is fixed for the task and for the entire dataset; all probabilities below are conditional on this realized pair unless stated otherwise. Symbols in $\mathcal{A} \setminus \mathcal{S}$ act as distractors and play no role in determining compliance.

Let P_0 denote the proposal distribution over candidate sequences: $X_t \stackrel{\text{iid}}{\sim} \text{Uniform}(\mathcal{A})$, $t = 1, \dots, n$. This i.i.d. uniform law is the base synthetic generation mechanism.

Definition 3.1. For lag parameter $\lambda \geq 1$, we say that X is *compliant* if there exists a chain of positions $(t_1, \dots, t_k)$ with $k \geq 2$ such that: **(i)** $t_{j+1} - t_j = \lambda$ for all j , **(ii)** $X_{t_j} \in \mathcal{S}$ for all j , **(iii)** $\kappa(X_{t_1}) \leq \dots \leq \kappa(X_{t_k})$.

Let $Y^* = \mathbf{1}\{X \text{ is compliant}\}$, let $\pi \in [0, 1/2)$ and let Y denote the observed label, generated from Y^* according to

$$P(Y = 1 \mid Y^* = y^*) = \begin{cases} 1 - \pi, & y^* = 1, \\ \pi, & y^* = 0, \end{cases} \quad (1)$$

In the experimental datasets of Sections 4 and 5, we subsample the draws from P_0 after labeling to obtain a controlled class balance. Thus the final benchmark distribution is a label-stratified subsample of P_0 , rather than P_0 itself. In general, after this balancing step, the coordinates of X need not remain independent and their marginals need not remain uniform. Theorem 3.3 below concerns the proposal law P_0 and, more generally, only requires independence and full support.

3.2 Pure Sequential Dependence

We use *pure sequential dependence* to refer to tasks whose outcome is determined exclusively by temporal ordering rather than by order-invariant statistics.

Definition 3.2 (Pure Sequential Dependence). A binary classification task exhibits *pure sequential dependence* if its outcome is determined exclusively by the temporal ordering of sequence elements and not by **(a)** individual element frequencies, **(b)** order-independent co-occurrence patterns, **(c)** aggregate sequence statistics, or **(d)** any other order-invariant sequence property.

A necessary, though not sufficient, single-position condition for pure sequential dependence is that individual symbols carry no information about the observed label. This can be quantified by the

190 mutual information

$$I(Y; X_t) = \sum_{y \in \{0,1\}} \sum_{a \in \mathcal{A}} P(Y=y, X_t=a) \times \log \frac{P(Y=y, X_t=a)}{P(Y=y) P(X_t=a)}. \quad (2)$$

191 We show that compliance tasks defined as 3.1 violate even this weakest necessary condition.

192 **Theorem 3.3** (Compliance Tasks Induce Local Information). *Fix a realized pair $(\mathcal{S}, \kappa)$ as in Section 3.1. Let $\lambda \geq 1$ and $n \geq \lambda + 1$. Let $X = (X_1, \dots, X_n)$ have independent coordinates with full support on $\mathcal{A}$, i.e. $P(X_t = a) > 0$ for all t and all $a \in \mathcal{A}$. Define $Y^* = \mathbf{1}\{X \text{ is compliant}\}$ and let Y be the observed label generated from Y^* according to (1). Let $\rho := P(Y^* = 1)$. If $\rho \in (0, 1)$, then there exist $a, b \in \mathcal{A}$ such that $P(Y^* = 1 \mid X_1 = a) \neq P(Y^* = 1 \mid X_1 = b)$. Consequently, $I(Y^*; X_1) > 0$. Moreover, $P(Y = 1 \mid X_1 = a) \neq P(Y = 1 \mid X_1 = b)$ and therefore $I(Y; X_1) > 0$.*

199 *Proof outline.* Let $s_{\min} \in \mathcal{S}$ satisfy $\kappa(s_{\min}) = 1$.

200 If $\mathcal{S} \subsetneq \mathcal{A}$, choose $a_0 \in \mathcal{A} \setminus \mathcal{S}$. Let $Z_2, \dots, Z_n$ be independent with $Z_t \sim \mathcal{L}(X_t)$, and define $\tilde{X} = (s_{\min}, Z_2, \dots, Z_n)$ and $\tilde{X}' = (a_0, Z_2, \dots, Z_n)$. Then $\tilde{X} \stackrel{d}{=} (X \mid X_1 = s_{\min})$ and $\tilde{X}' \stackrel{d}{=} (X \mid X_1 = a_0)$. Any compliant chain in $\tilde{X}'$ must avoid position 1, hence it is also compliant in $\tilde{X}$. On the positive-probability event that $Z_{1+\lambda} \in \mathcal{S}$ and $Z_t \notin \mathcal{S}$ for all $t \in \{2, \dots, n\} \setminus \{1 + \lambda\}$, the sequence $\tilde{X}$ is compliant via $(1, 1 + \lambda)$, while $\tilde{X}'$ is not. Therefore $P(Y^* = 1 \mid X_1 = s_{\min}) > P(Y^* = 1 \mid X_1 = a_0)$.

206 If instead $\mathcal{S} = \mathcal{A}$, then whenever $X_1 = s_{\min}$ the pair $(1, 1 + \lambda)$ is automatically compliant, giving $P(Y^* = 1 \mid X_1 = s_{\min}) = 1$. Since $\rho < 1$, there exists a^* with $P(Y^* = 1 \mid X_1 = a^*) < 1$, so $P(Y^* = 1 \mid X_1 = \cdot)$ is non-constant in both cases.

209 Finally, by (1), $P(Y = 1 \mid X_1 = a) = \pi + (1 - 2\pi) P(Y^* = 1 \mid X_1 = a)$. Because $\pi < 1/2$, the map $u \mapsto \pi + (1 - 2\pi)u$ is strictly increasing, so $P(Y = 1 \mid X_1 = \cdot)$ is non-constant as well, giving $I(Y^*; X_1) > 0$ and $I(Y; X_1) > 0$. See Appendix C.1 for the full proof. $\square$

212 Hence, non-degenerate compliance tasks under independent full-support sampling cannot satisfy pure sequential dependence, even in this single-position sense. This does not cover arbitrary order-sensitive binary labels. For example, if $\mathcal{A} = \{0, 1, 2, 3, 4\}$, $X_t \stackrel{\text{iid}}{\sim} \text{Uniform}(\mathcal{A})$, and $Y = \mathbf{1}\{(X_2 + 2X_3) \bmod 5 \in \{0, 1\}\}$, then $I(Y; X_t) = 0$ for every t , even though Y depends on the ordered pair (X_2, X_3) .

217 3.3 A Complementary Order-Invariant Global-Integration Benchmark

218 Theorem 3.3 shows that compliance tasks defined as in 3.1 inevitably leak local information. To study a task family with much stronger bounded-local invisibility, we introduce a parity-based construction.

220 **Definition 3.4** (Parity Task). *Fix a realized subset $K \subsetneq \mathcal{A}$ with $K \neq \emptyset$. For $X = (X_1, \dots, X_n)$ with $X_t \stackrel{\text{iid}}{\sim} \text{Uniform}(\mathcal{A})$, let $N_k := \sum_{t=1}^n \mathbf{1}\{X_t = k\}$ for each $k \in K$ and define $Y^* := |\{k \in K : N_k \text{ is even}\}| \bmod 2$.*

223 The quantity Y^* depends on the full sequence through a global parity computation over a hidden subset. Unlike the compliance family, this construction is not defined through ordered lagged chains.

225 **Theorem 3.5** (k -local purity of the parity task). *Assume the setting of Definition 3.4 with $p := |K|/|\mathcal{A}|$. Fix $n \geq 1$ and let $J \subseteq \{1, \dots, n\}$ with $1 \leq |J| < n$. Then $I(Y^*; X_J) = 0$ if $p = 1/2$, and $I(Y^*; X_J) > 0$ if $p \neq 1/2$. Moreover, if $p \neq 1/2$ and $J \subset \mathbb{N}$ is any fixed finite index set, interpreted as a subset of $\{1, \dots, n\}$ for all sufficiently large n , then $I(Y^*; X_J) \rightarrow 0$ as $n \rightarrow \infty$. If the observed label Y is generated from Y^* via (1) with $\pi < 1/2$, the same qualitative statements hold with Y in place of Y^* . In particular, when $p = 1/2$, the parity task is k -locally pure for every $k < n$.*

231 See Appendix C.3 for detailed proof.

232 Theorem 3.5 should be read as a bounded-local statement. In the balanced case $p = 1/2$, parity removes all local shortcuts supported on fewer than n positions. Outside the balanced case, finite- n

leakage is positive, although it vanishes for every fixed local view as n grows. Parity is therefore a useful global-integration stress test, but exact k -local purity is confined to the balanced regime.

4 Experiments

We evaluate whether different architectures can exploit sequential structure beyond what simple k -gram baselines capture.

4.1 Task Variants

We instantiate three task variants with different properties, see Appendix A for an example of each different mechanisms.

Naive setting ($m = \ell$, $\lambda = 1$, $\kappa = \text{natural order}$). Compliance requires the entire sequence to be non-decreasing in alphabetical order. The compliance probability is

$$\rho_{\text{naive}} = \binom{n + \ell - 1}{n} / \ell^n.$$

For $n = 20$, $\ell = 26$: $\rho_{\text{naive}} \approx 10^{-16}$ (see Appendix C), making the task degenerate without probabilistic labeling. Due to this extremely low compliance probability, we construct the naive dataset first generating alphabetically ordered (compliant) sequences and then creating non compliant counterparts by permuting selected elements within these sequences.

Tricky setting ($m \ll \ell$, $\lambda > 1$, $\kappa = \text{random but fixed}$). Only a small subset of letters matters, relevant letters must appear at specific lag intervals, and the ordering key is randomized. For $m = 6$, $\lambda = 7$, $n = 20$, $\ell = 26$, we obtain $\rho \approx 0.31$ via generation. This creates balanced classes while requiring models to: identify which letters matter, learn the hidden ordering, and track positions modulo the lag.

Parity setting Labels depend on the parity of key-letter occurrences. No local pattern carries information ($I(Y^*; X_t) = 0$ for all t), providing a test of pure global integration (see Theorem 3.5). For $m = 6$, $\lambda = 7$, $n = 20$, $\ell = 26$, we obtain $\rho \approx 0.5$ via generation.

4.2 Datasets

We instantiate the framework from Section 3 using the English alphabet ($\mathcal{A} = \{\text{A}, \dots, \text{Z}\}$, $\ell = 26$) with sequences of length $n = 20$. For both Tricky and Parity we used $\mathcal{S} = \{\text{W}, \text{D}, \text{Q}, \text{J}, \text{X}, \text{U}\}$ with $\kappa(\text{W}) < \kappa(\text{D}) < \kappa(\text{Q}) < \kappa(\text{J}) < \kappa(\text{X}) < \kappa(\text{U})$. Table 1 summarizes the three datasets. Each dataset is designed with a specific diagnostic purpose.

Table 1: Dataset characteristics for sequential classification tasks.

Dataset	N	n	ρ	λ	π	m
Naive	400K	20	0.51	–	0	26
Tricky det.	400K	20	0.293	7	0	6
Tricky random	400K	20	0.293	7	0.3	6
Parity	400K	20	0.499	–	0.0	6

The **tricky deterministic** dataset tests whether models can learn the sequential pattern without noise. The **tricky random** dataset introduces label noise ($\pi = 0.3$) and tests whether models plateau at k -gram levels or approach theoretical limits. The **parity** dataset tests a hard hidden-subset global-integration regime (Theorem 3.5). Although our $(m, \ell, n) = (6, 26, 20)$ setting is not exactly k -locally pure, the tested k -gram baselines remain at chance, so any success would have to come from beyond the local statistics captured by those baselines.

267 4.3 Models

268 We compare three categories of models, see Table 7 in Appendix D with a summary for all model
269 configurations.

270 **Pretrained LLMs (fine-tuned).** We evaluate Llama-3.2-1B, Llama-3.1-8B and Llama-3-70B [Tou-
271 vron et al., 2023, Grattafiori et al., 2024], fine-tuned for binary classification. We also include
272 Qwen3-4B-Think [Team, 2025] and Qwen2.5-14B [Team, 2024, Yang et al., 2024] and BERT-base
273 (110M parameters) [Devlin et al., 2019] with warm-start fine-tuning. These models assess whether
274 large-scale pretraining provides advantages for sequential pattern recognition.

275 **Architectures randomly initialized.** To isolate architectural capabilities from pretrained knowl-
276 edge, we train small-scale models from random initialization: Llama-style decoders (1M parameters),
277 LSTM [Hochreiter and Schmidhuber, 1997], Transformer encoder [Vaswani et al., 2017], and a
278 hybrid BiLSTM-Transformer [Graves and Schmidhuber, 2005], referred to as RNN-Transformer
279 throughout for conciseness, (100K–400K parameters each). These models disentangle architectural
280 effects from scale effects on sequential learning.

281 **Traditional baselines.** XGBoost [Chen and Guestrin, 2016] with two encoding strategies: posi-
282 tional encoding with integer-mapped characters (XGBoost-basic) and inspired by the works of Do
283 et al. [2023], Alshraideh et al. [2025] embeddings extracted from Llama-1B (XGBoost-LLM). These
284 baselines represent the state-of-the-art for tabular classification and test whether sequential structure
285 can be captured without recurrence or attention.

286 4.4 Sequence Encoding

287 We use three encoding strategies depending on the model family.

288 **Prompt-based encoding (LLMs and BERT).** For pretrained language models, we convert each
289 sequence into a natural language prompt. Given a sequence $(x_1, \dots, x_n)$, we construct:

290 Sequential events: $x_1 \ x_2 \ \cdots \ x_n$

291 For causal LLMs (Llama, Qwen), we append Outcome (0 or 1) to frame the task as next-token
292 prediction with a classification head on top. The prompt is then tokenized using each model’s
293 native tokenizer. This same prompt format is used to extract embeddings from Llama-3.2-1B for the
294 XGBoost-LLM baseline, where we apply mean pooling over the last hidden layer.

295 **Learned and ordinal embeddings.** For all non-pretrained models, we map each letter to an integer
296 index ($A \mapsto 1, B \mapsto 2, \dots, Z \mapsto 26$). For XGBoost-basic, this ordinal encoding is used directly as
297 features, relying on the tree ensemble to learn sequential dependencies. For neural architectures
298 (LSTM, Transformer, and hybrids), a learned embedding layer maps each index to a dense vector
299 $\mathbf{e}_i \in \mathbb{R}^{d_e}$ with $d_e \in \{32, 64, 128\}$. Transformer-based models additionally incorporate learnable
300 positional encodings $\mathbf{h}_i = \mathbf{e}_i + \mathbf{p}_i$, where $\mathbf{P} \in \mathbb{R}^{L_{\max} \times d_e}$ is initialized from a standard normal
301 distribution. Sequences are zero-padded within each mini-batch.

302 4.5 k -gram Baseline

303 A k -gram is a contiguous subsequence of k consecutive symbols within a sequence: given $X =$
304 $(X_1, \dots, X_n)$, the k -grams are $(X_t, X_{t+1}, \dots, X_{t+k-1})$ for $t = 1, \dots, n - k + 1$. For instance, the
305 sequence ABCD contains the 2-grams AB, BC, CD. The k -gram classifier provides a reference for local
306 pattern exploitation:

307 **Training.** For each k -gram g in the training data, compute:

$$\hat{\mathbb{P}}(Y = 1 \mid g) = \frac{\#(g, Y = 1)}{\#(g)}. \quad (3)$$

308 **Inference.** For test sequence X , extract all k -grams $\{g_1, \dots, g_{n-k+1}\}$ and predict the average
 309 probability:

$$\hat{\mathbb{P}}(Y = 1 | X) = \frac{1}{n - k + 1} \sum_{j=1}^{n-k+1} \hat{\mathbb{P}}(Y = 1 | g_j). \quad (4)$$

310 The four datasets differ in their k -gram distributions due to distinct generation procedures. For
 311 the tricky and parity tasks, sequences were generated via brute-force sampling (uniform 1/26
 312 probability per letter), and then classified by the correspondent ordering criteria. For the Naive task,
 313 we followed the approach described in 4.1. This results in higher k -gram informativeness on the
 314 outcome across train/validation/test splits for naive compared to other datasets. In fact, in the naive
 315 setting, bigrams alone ($k = 2$) carry strong signal (e.g., certain bigrams are “forbidden” in ordered
 316 sequences) enabling immediate classification (AUC $\sim$ 0.999 just with a bigram counting model). As
 317 task complexity increases, this local signal diminishes (see Appendix B for detailed analysis).

318 4.6 Training Procedure

319 All neural models are trained with AdamW [Loshchilov and Hutter, 2019], cross-entropy loss, and
 320 early stopping on validation performance. Pretrained LLMs (Llama, Qwen) and BERT are fine-tuned
 321 using LoRA [Hu et al., 2021] for parameter efficiency, with gradient checkpointing and mixed-
 322 precision training for larger models. Randomly initialized models (LSTM, BiLSTM, Transformer)
 323 use architecture-specific learning rates and dropout. XGBoost baselines use regularized gradient
 324 boosting with class-weighted sampling. For all models, we optimize the classification threshold on
 325 the validation set to maximize F1 score. Experiments run on a single NVIDIA A100 GPU with three
 326 random seeds; we report means and standard deviations. Full hyperparameter configurations are
 327 provided in Appendix D. Code will be made publicly available upon acceptance.

328 4.7 Theoretical Performance Bounds

329 We establish the optimal achievable performance, which serves as a ceiling for model evaluation.

330 **Theorem 4.1** (Optimal Performance under Label Noise). *Under symmetric label noise with parameter*
 331 $\pi \in [0, 0.5)$ *and compliance probability* ρ *(i.e., $\mathbb{P}(Y^* = 1) = \rho$, and $\mathbb{P}(Y = 1 | Y^* = 1) = 1 - \pi$),*
 332 *a classifier with perfect access to compliance status achieves:*

$$\text{AUC}^* = \frac{1}{2} + \frac{1}{2}(A - C) \stackrel{\rho=0.5}{=} 1 - \pi$$

333 where

$$A = \frac{(1 - \pi)\rho}{\rho(1 - \pi) + (1 - \rho)\pi}, \quad C = \frac{\pi\rho}{\rho\pi + (1 - \rho)(1 - \pi)}.$$

334 Similarly, $F_1^* = 2(1 - \pi)\rho / (\pi + 2\rho(1 - \pi)) \stackrel{\rho=0.5}{=} 1 - \pi$.

335 For $\pi = 0.3$ (30% label noise) and $\rho = 0.5$, this gives an optimal $\text{AUC}^* = F_1^* = 0.70$. See
 336 Appendix C for the full proof. In Table 2 are shown for each dataset the optimal performance metrics.

Table 2: Optimal performance achievable by a perfect classifier for each dataset.

Dataset	AUC*	F1*	Prec*	Rec*
Naive	1.000	1.000	1.000	1.000
Tricky Det.	1.000	1.000	1.000	1.000
Tricky Random	0.670	0.577	0.700	0.491
Parity	1.000	1.000	1.000	1.000

337 4.8 Evaluation Protocol

338 We report AUC, F1, Precision, and Recall on held-out test sets (20% of data). For all datasets, we
 339 compare against the theoretical optimum (Theorem 4.1, indicated by symbol *). We vary training set
 340 size from 4K to 400K samples to assess data efficiency. If (1) $\text{AUC} \approx 0.50$ that means the model

Model Comparison: AUC Across Datasets

Test AUC by training set size (shaded: estimated variability across runs)

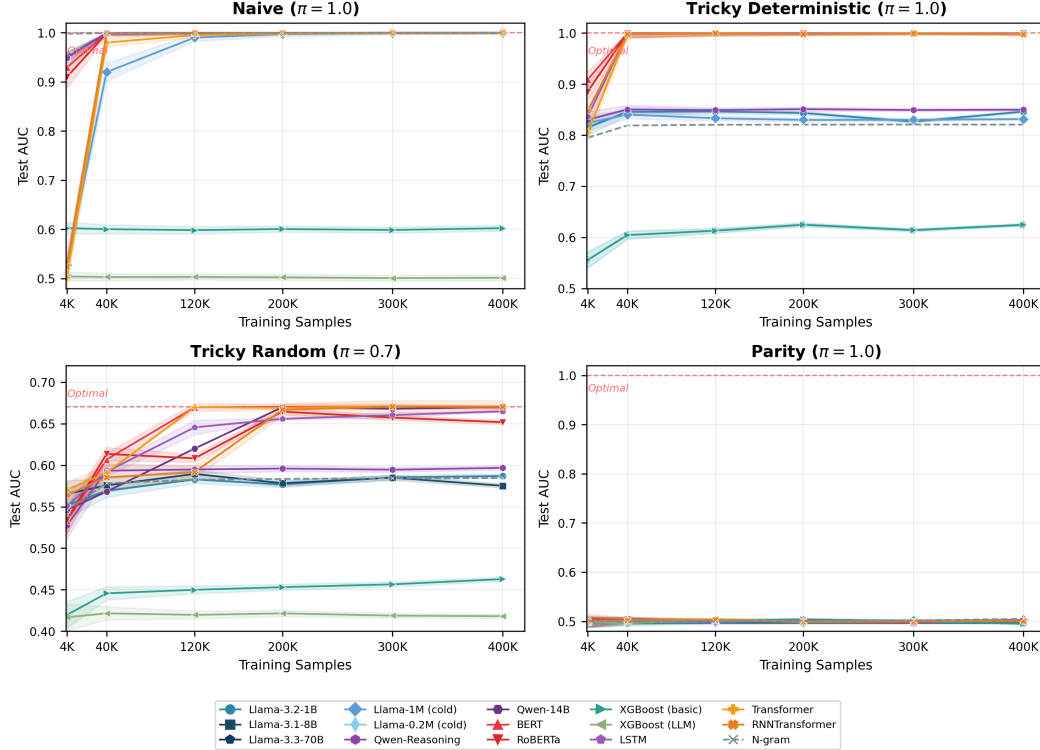


Figure 1: Test AUC across four synthetic datasets. **Naive**: all models except XGBoost solve the task. **Tricky Deterministic**: bidirectional encoders, recurrent models, and larger causal decoders (8B) reach optimality, while smaller decoders (1B) plateau at ~ 0.85 . **Tricky Random**: bidirectional models approach the optimal bound (0.67) with modest parameter counts, while decoder-only models require substantially more parameters (14B) to reach similar performance. **Parity**: no model exceeds chance, including Llama-70B. Shaded regions indicate variability across runs.

learns nothing about compliance (2) $0.55 < \text{AUC} < \text{AUC}^*$: partial recovery of the sequential signal
 (3) $\text{AUC} \approx \text{AUC}^*$: full recovery, limited only by label noise. We adopt AUC as our primary metric
 since it captures a model’s discriminative capacity without relying on arbitrary threshold selection.

5 Results

We evaluate all models across the four synthetic datasets described in Table 1. For a summary see Table 3 and Figure 1 which present an overview of our findings. Full results are reported and shown in Appendix E.

Naive setting: (almost) all methods succeed. When discriminative signal resides in simple local co-occurrences (k -grams), almost all approaches achieve near-perfect classification. The notable exception is XGBoost: despite preserving positional information through ordinal encoding, tree-based methods struggle because verifying sequential order requires *pairwise comparisons* between adjacent positions. Gradient boosted trees make splits on individual features, making such relational dependences difficult to catch without explicit feature engineering or architectural modifications [Marton and Schneider, 2025]. Neural architectures, by contrast, naturally capture sequential dependencies through attention or recurrence.

Tricky deterministic: encoders excel, decoders lag. When the task requires detecting ordered subsequences with specific positional constraints ($\lambda = 7$), a clear architectural segmentation emerges. Encoder-based models (BERT, RoBERTa) and bidirectional architectures (LSTM, Transformer) achieve near-perfect performance ($\text{AUC} \geq .997$). In contrast, decoder-only LLMs show variable results: Llama-3.1-8B reaches AUC .995, but Llama-3.2-1B plateaus at .846 which is comparable to the simple 2-gram baseline (.821). Notably, XGBoost with LLMs embeddings is the *worse*, suggesting that frozen embeddings discard the sequential structure needed for this task.

Tricky random: theoretical ceiling approached. The stochastic variant ($\pi = 0.3$) has a theoretical AUC upper bound of .670 due to inherent label noise. Our custom Transformer and RNN-Transformer hybrid both reach this ceiling (AUC .671), demonstrating that the architectures can extract maximal signal when trained appropriately. BERT approaches the optimum (.669), while LSTM achieves .665. Smaller decoder-only models (Llama-1B, Llama-8B) plateau around .58; only marginally above the 2-gram baseline (.585). However, at sufficient scale (Qwen-14B), decoder-only architectures also approach the theoretical ceiling, suggesting that autoregressive models can learn global sequential dependencies when provided with enough capacity (and data).

Parity: failure under standard supervised training. The parity task presents a stark empirical result: every model performs at chance level ($\text{AUC} \approx .50$), regardless of architecture or scale. This includes 70B-parameter LLMs, bidirectional transformers, and recurrent networks. In our hidden-subset parity regime, success requires jointly identifying the hidden subset, filtering distractors, and computing parity over the relevant counts. The tested k -gram baselines also fail. We therefore interpret this as a failure of the standard supervised training pipeline in this regime rather than as a universal impossibility result for transformers or for parity-like tasks more broadly.

Signal decomposition: content vs. order. To quantify how much predictive signal arises from order-invariant letter frequencies versus sequential structure, we introduce a hierarchy of baselines (Table ??). *BoE-key* trains a logistic regression on only the counts of the six key letters $\mathcal{S}$ (a 6-dimensional feature vector), capturing content signal from the task-relevant subset alone. *BoE-XGB* trains a gradient-boosted tree (XGBoost) on the full 26-dimensional letter-count vector, serving as a nonlinear content-only ceiling that exploits all letter-frequency information while remaining completely order-invariant. The *best k -gram* row takes the strongest position-aware k -gram classifier from Table 4 ($k=2$ for both tricky variants), which can capture short-range local order beyond mere letter counts. *Best model* is the highest AUC achieved by any neural architecture (from Table 15).

On Tricky Deterministic, content alone accounts for 63% of the total signal ($\Delta_{\text{content}} = +0.316$), while sequential modeling contributes the remaining 37% ($\Delta_{\text{seq}} = +0.178$). On Tricky Random, the split is approximately 50/50: content yields AUC 0.584, and sequential modeling adds +0.086 to reach the theoretical ceiling. Crucially, even the best k -gram adds negligible signal beyond content ($\Delta_{\text{local}} \leq 0.005$), confirming that the models’ advantage over BoE comes from richer sequential structure, not local k -gram patterns. A matched-histogram counterfactual reinforces this (Table ??): when sequences are grouped by their key-letter count vector, 94–96% fall into ambiguous groups where both labels are present, and BoE achieves only AUC 0.799 (Det.) and 0.578 (Rnd.) on this subset. Parity provides an instructive contrast: zero ambiguous groups exist because the label is fully determined by whether the total count of key letters is even or odd, every key-letter count vector maps to a single label, so parity is not a sequential task at all. Yet all models, including a nonlinear BoE (XGBoost) with direct access to the relevant counts, fail ($\text{AUC} \approx 0.50$). This confirms that parity failure reflects the computational hardness of modular arithmetic over a hidden subset, not missing sequential information, a complementary explanation to local-pattern reliance.

6 Discussion

Our experiments reveal a clear hierarchy of sequential exploitation capabilities across architectures, with implications for both theoretical understanding and practical applications.

The parity barrier. The strongest negative result in the paper is the complete failure of all tested models on hidden-subset parity under standard supervised training, in line with the work of Fu et al. [2024]. Theorem in Section 3.3 clarifies the theoretical role of this family: exact k -local purity holds only in the balanced case $|K|/|\mathcal{A}| = 1/2$, whereas our experimental regime lies outside that exact

Table 3: Summary of model performance across tasks, using criteria from Section 4.8. ✓: full recovery ($AUC \approx AUC^*$); ~: partial recovery ($0.55 < AUC < AUC^*$); ×: no learning ($AUC \approx 0.50$). Underlined: most data-efficient models to reach the optimum. All evaluations based on 100% training data.

Model	Naive	Tr. Det.	Tr. Rnd.	Parity
k-gram baseline	✓	~	~	×
XGBoost	~	~	×	×
<u>BERT</u>	✓	✓	✓	×
<u>RoBERTa</u>	✓	✓	✓	×
Llama-3.2-1B	✓	~	~	×
Qwen3-4B-Think	✓	~	~	×
Llama-3.1-8B	✓	✓	~	×
Qwen2.5-14B	✓	✓	✓	×
Llama-3.1-70B	✓	✓	✓	×
LSTM	✓	✓	✓	×
<u>Transformer</u>	✓	✓	✓	×
RNN-Transformer*	✓	✓	✓	×

*We call RNN-Transformer a BiLSTM-Transformer.

408 purity regime but still defeats the tested k -gram baselines. This should not be read as a universal
 409 statement that parity-type tasks are unlearnable or that transformers cannot implement parity in
 410 simpler settings. Binary-vocabulary parity is known to be learnable +++CITE?+++. Rather, our result
 411 isolates a harder compositional regime in which subset identification, distractor filtering, and parity
 412 computation must be learned jointly from end-to-end supervision.

413 More broadly, failure of fixed k -gram baselines does not imply failure of transformers in general.
 414 Associative-recall mechanisms of induction-head type provide one important counterexample. Our
 415 empirical claim is narrower: on the specific hidden-subset parity benchmark studied here, neither
 416 the tested k -gram baselines nor the tested neural architectures recover useful signal under standard
 417 supervised training.

418 **Scale unlocks sequential learning in decoder-only models.** A key finding is that decoder-only
 419 models *can* approach the theoretical optimum on the tricky random task, but only at sufficient
 420 scale. While Llama-1B and Llama-8B plateau around $AUC \approx 0.58$ (comparable to the k -gram
 421 baseline), Qwen-14B approaches the theoretical ceiling of 0.67. This suggests that autoregressive
 422 architectures are not fundamentally limited in their ability to capture global sequential dependencies,
 423 but require substantially more parameters (and data) to do so. Notably, while Qwen-14B achieves
 424 near-optimal discrimination (AUC), its F1 score is still increasing at 400K training samples (see
 425 Figure 8), indicating that the model has learned to correctly rank positive and negative examples
 426 but has not yet fully calibrated its predictions. This suggests that even larger training sets may be
 427 needed to translate discriminative capacity into optimal predictive performance. The contrast with
 428 encoder-based architectures is instructive: a custom Transformer with $\sim 100K$ parameters achieves
 429 the same performance as a 14B-parameter decoder. Rather than a limitation of decoder-only models,
 430 this gap indicates that next-token prediction requires greater capacity to learn classification-relevant
 431 sequential patterns. This finding nuances recent observations by Nguyen [2024] and Kim et al. [2025]
 432 that LLMs often reduce to local pattern matching. Our results suggest this deficiency can be overcome
 433 with sufficient scale, though the computational cost may be prohibitive for many applications.

434 **Architecture and scale interact.** The architectural comparison reveals a nuanced picture. Encoder-
 435 based models (BERT, Transformers) and recurrent architectures (LSTM, RNN-Transformer) approach
 436 the theoretical optimum with relatively few parameters. Decoder-only models can also reach this ceiling,
 437 but require substantially more capacity and more data. This reflects a fundamental mismatch: the
 438 causal attention mask forces autoregressive models to encode sequential patterns through parameters
 439 rather than bidirectional integration.

440 Despite these challenges, however, our results reveal substantial potential. On tasks deliberately
 441 designed to eliminate semantic content, encoder-based transformers and sufficiently scaled decoders
 442 still approach theoretical optima, suggesting these models develop internal representations capable
 443 of capturing sequential structure even without semantic scaffolding. The scaling trends we observe

444 point toward a promising direction: leveraging the rich representations within LLMs for sequential
445 tasks where not just order but also meaning *does* matter, combining their pattern-learning capabilities
446 with their vast pretrained knowledge.

447 **Limitations and future directions.** Our study has several limitations that suggest directions for
448 future work. We tested sequences of fixed length ($n = 20$) and alphabet size ($\ell = 26$) and $\lambda = 7$,
449 when used; examining how performance scales with sequence length could reveal whether the encoder-
450 decoder gap widens or narrows for longer contexts. While we tested models up to 14B parameters
451 (Llama-70B only on parity), larger models might exhibit additional scaling effects. Our synthetic
452 tasks, while controlled, may not capture all aspects of real-world sequential data extending our
453 framework to clinical trajectories or other temporal data could validate whether these patterns transfer
454 to practice. The efficiency gap between encoders and decoders may also narrow in settings where
455 semantic understanding complements sequential structure. Finally, we focused on classification;
456 generative tasks might engage different capabilities. Other promising directions include mechanistic
457 interpretability analyses to clarify what features models actually learn, and architectural modifications
458 such as auxiliary objectives or structural inductive biases that explicitly target global integration.

459 AI Disclosure

460 AI tools (LLMs) were used for code debugging, grammar improvement, and visualization scripts.
461 They were **not** used for conducting research or generating scientific ideas. The authors take full
462 responsibility for the content.

References

- Abedalrahman Alshraideh, Bayan Al Fayoumi, Bahaaldeem M Alshraideh, and Mohammad Alshraideh. Hybrid ai framework for the early detection of heart failure: integrating traditional machine learning and generative language models with clinical data. *Cureus*, 17(6):e85638, 2025.
- Thomas Beaney, Sneha Jha, Asem Alaa, Alexander Smith, Jonathan Clarke, Thomas Woodcock, Azeem Majeed, Paul Aylin, and Mauricio Barahona. Comparing natural language processing representations of coded disease sequences for prediction in electronic health records. 31(7): 1451–1462, 2024. ISSN 1067-5027, 1527-974X. doi: 10.1093/jamia/ocae091. URL <https://academic.oup.com/jamia/article/31/7/1451/7667337>.
- Suhana Bedi, Yixing Jiang, Philip Chung, Sanmi Koyejo, and Nigam Shah. Fidelity of Medical Reasoning in Large Language Models. *JAMA Network Open*, 8(8):e2526021, August 2025. ISSN 2574-3805. doi: 10.1001/jamanetworkopen.2025.26021.
- Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of KDD*, pages 785–794, 2016.
- Hejie Cui, Alyssa Unell, Bowen Chen, Jason Alan Fries, Emily Alsentzer, Sanmi Koyejo, and Nigam Shah. Timer: Temporal instruction modeling and evaluation for longitudinal clinical records, 2025. URL <https://arxiv.org/abs/2503.04176>.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL*, pages 4171–4186, 2019.
- Tuyen Do, Bichar Dip Shrestha Gurung, Shiva Aryal, Anup Khanal, Sandeep Chataut, Venkataramana Gadhamshetty, Carol Lushbough, and Etienne Z. Gnimpieba. Utilizing xgboost for the prediction of material corrosion rates from embedded tabular data using large language model. In *2023 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*, pages 4497–4499, 2023. doi: 10.1109/BIBM58861.2023.10385544.
- Brice Edelman, Hannah Kim, and Jeffrey Skolnick. Scalable discovery and validation of order-specific electronic health record event trajectories for interpretable adverse-outcome risk estimation. *Journal of Biomedical Informatics*, 177:105008, May 2026. ISSN 15320464. doi: 10.1016/j.jbi.2026.105008.
- Bahare Fatemi, Mehran Kazemi, Anton Tsitsulin, Karishma Malkan, Jinyeong Yim, John Palowitch, Sungyong Seo, Jonathan Halcrow, and Bryan Perozzi. Test of time: A benchmark for evaluating llms on temporal reasoning, 2024. URL <https://arxiv.org/abs/2406.09170>.
- Tairan Fu, Raquel Ferrando, Javier Conde, Carlos Arriaga, and Pedro Reviriego. Why do large language models (llms) struggle to count letters?, 2024. URL <https://arxiv.org/abs/2412.18626>.
- Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020.
- Aaron Grattafiori, Abhimanyu Dubey, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Alex Graves and Jürgen Schmidhuber. Framewise phoneme classification with bidirectional lstm and other neural network architectures. *Neural Networks*, 18(5-6):602–610, 2005.
- Albert Gu and Tri Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces, 2024. URL <http://arxiv.org/abs/2312.00752>.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8): 1735–1780, 1997.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021. URL <https://arxiv.org/abs/2106.09685>.

512 Daniel Kaiser, Arnaldo Frigessi, Ali Ramezani-Kebrya, and Benjamin Ricaud. Cogniload: A
513 synthetic natural language reasoning benchmark with tunable length, intrinsic difficulty, and
514 distractor density. *arXiv preprint arXiv:2509.18458*, 2025.

515 Sein Kim, Hongseok Kang, Kibum Kim, Jiwan Kim, Donghyun Kim, Minchul Yang, Kwangjin Oh,
516 Julian McAuley, and Chanyoung Park. Lost in Sequence: Do Large Language Models Understand
517 Sequential Recommendation? In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge
518 Discovery and Data Mining V.2*, pages 1160–1171, Toronto ON Canada, August 2025. ACM.
519 ISBN 979-8-4007-1454-2. doi: 10.1145/3711896.3737035.

520 Anton Klenitskiy, Anna Volodkevich, Anton Pembek, and Alexey Vasilev. Does It Look Sequential?
521 An Analysis of Datasets for Evaluation of Sequential Recommendations. In *18th ACM Conference
522 on Recommender Systems*, pages 1067–1072, October 2024. doi: 10.1145/3640457.3688195.

523 Yikuan Li, Shishir Rao, Jose Roberto Ayala Solares, et al. Behrt: Transformer for electronic health
524 records. *Scientific Reports*, 10:7155, 2020.

525 Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019. URL <https://arxiv.org/abs/1711.05101>.

527 Sascha Marton and Moritz Schneider. Decision trees that remember: Gradient-based learning of
528 recurrent decision trees with memory, 2025. URL <https://arxiv.org/abs/2502.04052>.

529 R Thomas McCoy, Ellie Pavlick, and Tal Linzen. Right for the wrong reasons: Diagnosing syntactic
530 heuristics in natural language inference. In *Proceedings of ACL*, pages 3428–3448, 2019.

531 Iman Mirzadeh, Keivan Alizadeh, Hooman Shahrokhi, Oncel Tuzel, Samy Bengio, and Mehrdad
532 Farajtabar. Gsm-symbolic: Understanding the limitations of mathematical reasoning in large
533 language models. *arXiv preprint arXiv:2410.05229*, 2024.

534 Timothy Nguyen. Understanding transformers via n-gram statistics, 2024. URL <https://arxiv.org/abs/2407.12034>.

536 Catherine Olsson, Nelson Elhage, Neel Nanda, et al. In-context learning and induction heads. *arXiv
537 preprint arXiv:2209.11895*, 2022.

538 Davide Placido, Bo Yuan, Jessica X. Hjaltelin, Chunlei Zheng, Amalie D. Haue, Piotr J. Chmura,
539 Chen Yuan, Jihye Kim, Renato Umeton, Gregory Antell, Alexander Chowdhury, Alexandra Franz,
540 Lauren Brais, Elizabeth Andrews, Debora S. Marks, Aviv Regev, Siamack Ayandeh, Mary T.
541 Brophy, Nhan V. Do, Peter Kraft, Brian M. Wolpin, Michael H. Rosenthal, Nathanael R. Fillmore,
542 Søren Brunak, and Chris Sander. A deep learning algorithm to predict risk of pancreatic cancer from
543 disease trajectories. 29(5):1113–1122, 2023. ISSN 1546-170X. doi: 10.1038/s41591-023-02332-5.
544 URL <https://www.nature.com/articles/s41591-023-02332-5>.

545 Freya Pollington, Spiros C Denaxas, Kezhi Li, Johan H Thygesen, Georgios Lyratzopoulos, and
546 Becky White. Evaluation of trajectory analysis for disease risk assessment: A scoping review. 33
547 (2):521–535, 2026. ISSN 1527-974X. doi: 10.1093/jamia/ocaf208. URL <https://doi.org/10.1093/jamia/ocaf208>.

549 Mohammad Ramezanali, Mo Vazifeh, and Paolo Santi. seqBench: A Tunable Benchmark to Quantify
550 Sequential Reasoning Limits of LLMs, September 2025.

551 Laila Rasmy, Yang Xiang, Ziqian Xie, Cui Tao, and Degui Zhi. Med-bert: pretrained contextualized
552 embeddings on large-scale structured electronic health records for disease prediction. *npj Digital
553 Medicine*, 4:86, 2021.

554 Kenneth H. Rosen. *Discrete Mathematics and Its Applications: And Its Applications*. McGraw-Hill
555 Higher Education, 2006. ISBN 0072880082.

556 Germans Savcisens, Tina Eliassi-Rad, Lars Kai Hansen, Laust Hvas Mortensen, Lau Lilleholt,
557 Anna Rogers, Ingo Zettler, and Sune Lehmann. Using sequences of life-events to predict human
558 lives. *Nature Computational Science*, 4(1):43–56, December 2023. ISSN 2662-8457. doi:
559 10.1038/s43588-023-00573-5. URL <http://dx.doi.org/10.1038/s43588-023-00573-5>.

560 Artem Shmatko, Alexander Wolfgang Jung, Kumar Gaurav, Søren Brunak, Laust Hvas Mortensen,
561 Ewan Birney, Tom Fitzgerald, and Moritz Gerstung. Learning the natural history of human disease
562 with generative transformers. *Nature*, 647(8088):248–256, November 2025. ISSN 1476-4687. doi:
563 10.1038/s41586-025-09529-3.

564 Qwen Team. Qwen2.5: A party of foundation models, September 2024. URL [https://qwenlm.](https://qwenlm.github.io/blog/qwen2.5/)
565 [github.io/blog/qwen2.5/](https://qwenlm.github.io/blog/qwen2.5/).

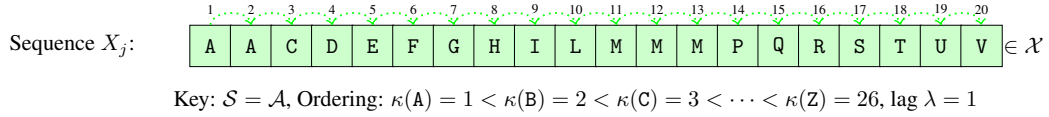
566 Qwen Team. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.

567 Hugo Touvron, Thibaut Lavril, Gautier Izacard, et al. Llama: Open and efficient foundation language
568 models. *arXiv preprint arXiv:2302.13971*, 2023.

569 Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz
570 Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing*
571 *Systems*, 30, 2017.

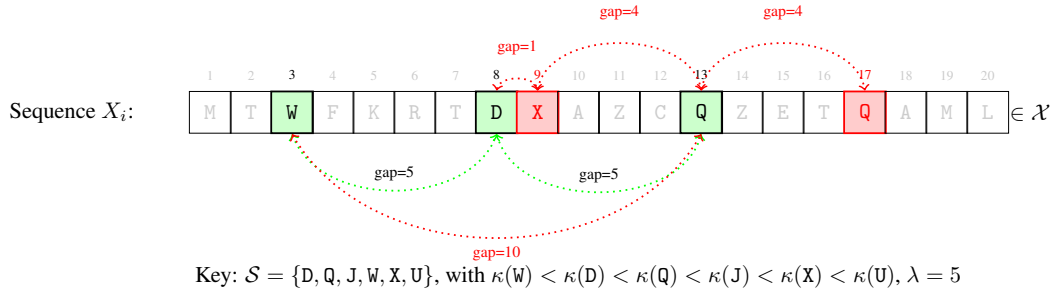
572 An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li,
573 Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang,
574 Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jin Xu, Jingren Zhou, Jinze Bai,
575 Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng
576 Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai
577 Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan
578 Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Yang Fan, Yang Yao, Yichang
579 Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zhihao Fan. Qwen2
580 technical report. *arXiv preprint arXiv:2407.10671*, 2024.

A Ordering Mechanisms



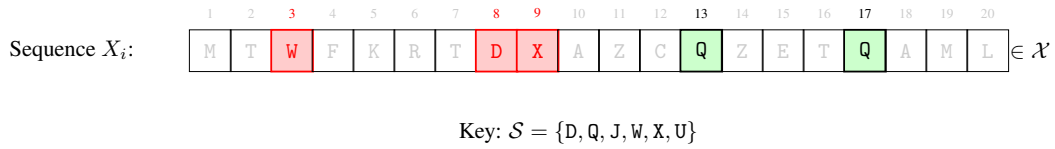
The subsequence consisting on the whole sequence satisfies both lag $\lambda=1$ and the ordering key $\kappa(\mathcal{S}) \equiv \mathcal{A}$.
 $\Rightarrow$ Ordered (i.e., $Y^* = 1$)

Figure 2: Sequence classification example using the *naive* method. Key letters $\mathcal{S} = \mathcal{A}$ (i.e., all letters). Arrows show gaps between key letters. In green letters (and gaps) that do have lag-connections with other key letters and that satisfy the ordering $\kappa(\mathcal{S}) \equiv \mathcal{A}$. Note: in this example $n = 20$, $m = 26$.



The subsequence W-D-Q satisfies both lag $\lambda=5$ and the ordering key $\kappa(\mathcal{S}) \Rightarrow$ Ordered (i.e., $Y^* = 1$)

Figure 3: Sequence classification with *tricky* method. Key letters $\mathcal{S} = \{D, Q, J, W, X, U\}$ are extracted from the full sequence (non-key in gray). Arrows indicate gaps. Red: letters/gaps without lag-connections to other keys. Green: letters/gaps with lag-connections satisfying ordering $\kappa(\mathcal{S})$. Here $n = 20$, $m = 4$.



An odd number of letters belonging to the Key appear in an even manner in the sequence
 $\Rightarrow$ Not Compliant (i.e., $Y^* = 0$)

Figure 4: Sequence classification example using the *parity* method. Key letters $\mathcal{S} = \{D, Q, J, W, X, U\}$ are extracted from the full sequence (non-key letters in gray). The parity criterion requires an *even number of unique* key letter to appear an **even** number of times. In green: key letters with even count. In red: key letters with odd count. Since not all key letters have even counts, the sequence is labeled as non-compliant ($Y^* = 0$).

B K-grams Analysis

This appendix provides detailed k -gram statistics for our benchmark datasets. We report two complementary measures: *coverage* and *predictive capacity*.

Coverage quantifies the proportion of k -grams in the test set that also appear in the training set. High coverage indicates that models can rely on patterns seen during training; low coverage suggests generalization to unseen local patterns is required.

Predictive capacity measures how well k -gram presence alone predicts the outcome, quantifying the extent to which the task is exploitable through local shortcuts rather than global sequential processing.

For the Tricky and Parity datasets, these statistics are identical, as both were generated through the same brute-force sampling procedure (see Section 4.1). The Naive dataset, however, required a different simulation strategy due to the low probability of randomly generating ordered sequences. This resulted in a distinct k -gram distribution and, notably, higher predictive capacity at low k , reflecting the presence of “forbidden” bigrams that immediately reveal sequence ordering.

Tables 5, 5 and 6 report these measures for $k \in \{1, \dots, 7\}$ (for the naive setting we show until $k = 20$).

Table 4: k -gram baseline predictive performance across datasets. Results shown for optimized classification thresholds. *Tricky Deterministic*: Deterministic sequence with statistical shortcuts. *Tricky Random*: Same structure with added noise ($\pi = 0.3$). *Parity*: Parity task requiring full sequence context.

k	Cov. Test	Tricky Deterministic				Tricky Random				Parity			
		AUC	F1	Prec	Rec	AUC	F1	Prec	Rec	AUC	F1	Prec	Rec
1	100.0%	.813	.645	.665	.626	.585	.593	.422	.996	.502	.668	.502	1.00
2	100.0%	.819	.669	.587	.777	.586	.593	.422	.998	.498	.668	.502	1.00
3	100.0%	.818	.669	.591	.772	.576	.593	.423	.993	.497	.668	.502	1.00
4	100.0%	.781	.639	.531	.801	.534	.593	.421	1.00	.499	.668	.502	1.00
5	41.67%	.617	.539	.383	.907	.508	.593	.421	1.00	.497	.668	.502	1.00
6	1.9%	.524	.528	.359	.999	.499	.593	.421	1.00	.501	.668	.502	1.00
7	0.1%	.501	.528	.358	1.00	.501	.593	.422	1.00	.500	.668	.502	1.00

Table 5: k -gram coverage between training and test sets. Analysis is common across tricky deterministic, tricky random and parity datasets as they share identical sequences. For the naive, see table below.

k	Unique Train	Unique Test	Coverage
1	26	26	100.0%
2	676	676	100.0%
3	17,576	17,576	100.0%
4	456,976	385,868	100.0%
5	4,948,910	773,862	41.7%
6	5,942,184	749,024	1.9%
7	5,598,066	699,963	0.1%

Note: Coverage represents the percentage of test k -grams that appear in the training set. Sharp drop at $n \geq 5$ indicates severe data sparsity, explaining baseline model collapse in Table 4.

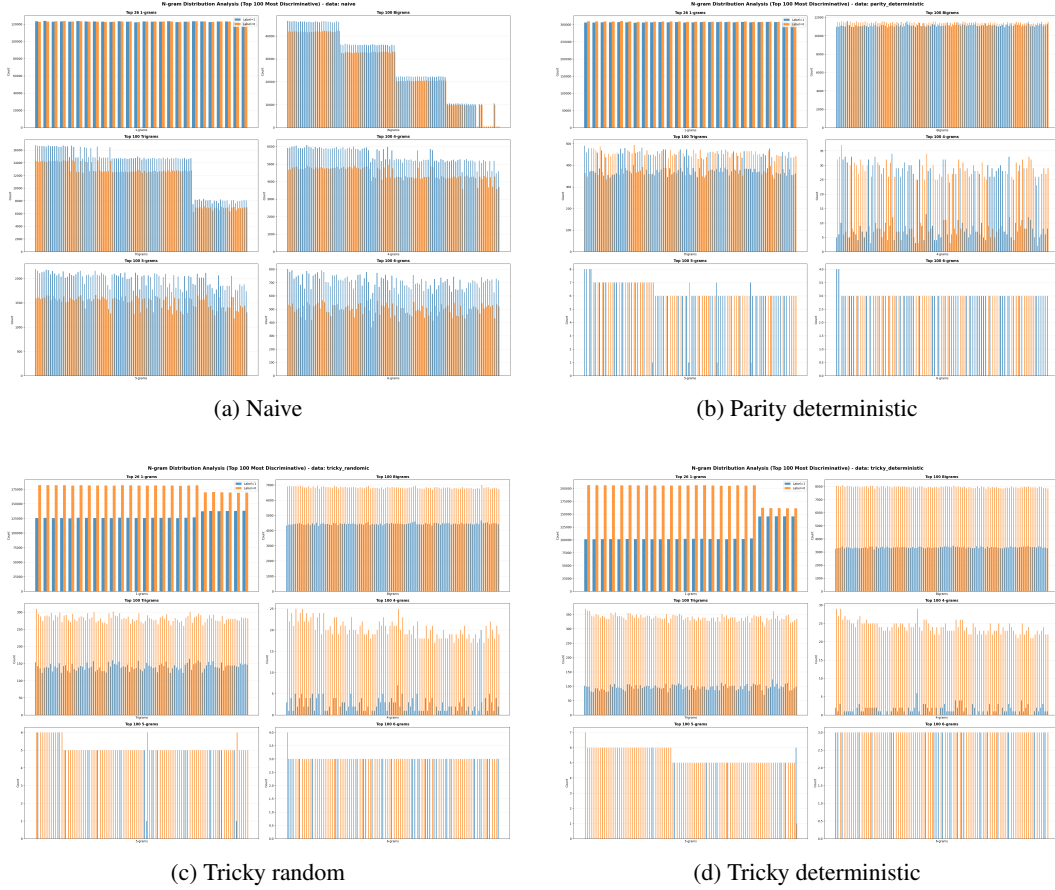


Figure 5: Frequency distribution of the top-100 most discriminative k -grams ($k \in \{1, 2, 3, 4, 5, 6\}$) by class label across the four synthetic datasets (*blue* is ordered (i.e. Label=1), *orange* unordered (i.e. Label=0)). In the *naive* setting (a), discriminative signal concentrates in bigrams, with large frequency differences between classes and high absolute counts. Notably, disordered pairs produce *illegal bigrams* that never appear in ordered sequences, providing an extremely strong signal for k -gram classifiers. The *parity* methods (b) show a qualitatively different pattern. While some k -grams (particularly for $k = 6$) appear exclusively for one class, visible as bars present for only one label, two factors limit their utility. First, these discriminative k -grams occur with very low absolute frequencies (often ≤ 10 occurrences across $> 400,000$ training examples). Second, there is a coverage problem: higher-order k -grams observed in training may simply not appear in test sequences. Lower-order k -grams (unigrams, bigrams) have inherently higher frequencies and coverage, being shorter, they are more likely to recur across both training and test sets. This creates a form of inherent “data leakage”: not leakage due to methodological error, but a structural property of k -gram statistics where shorter patterns naturally generalize better, regardless of whether they capture the true underlying sequential mechanism. In contrast, *tricky* methods (c, d) exhibit strong discriminative signal already at the unigram level: Label=0 sequences contain substantially more letter occurrences overall due to the outcome-dependent generation process, with the deterministic variant (d) showing the most pronounced class separation ($\sim 100k$ difference in 1-gram frequencies).

Table 6: K-gram analysis for the *naive* dataset demonstrating information leakage through local patterns. At $k = 20$, coverage drops to 0%: no identical sequences exist between train and test sets, ruling out traditional data leakage. Yet discriminative signal persists at small k : bigrams achieve near-perfect AUC (0.999), confirming that ordering-based tasks leak information through local statistics (Theorem 3.3). The AUC decrease for $k \geq 7$ reflects coverage sparsity rather than absence of signal.

k	AUC	Unique Train	Unique Test	Coverage
1	0.493	26	26	100.0%
2	0.999	676	676	100.0%
3	0.999	11,125	8,966	99.1%
4	0.998	66,032	34,478	94.9%
5	0.985	231,440	76,714	85.5%
6	0.906	526,836	135,307	72.2%
7	0.667	895,093	219,902	66.8%
8	0.383	1,344,547	316,809	66.5%
9	0.211	1,859,350	380,617	61.9%
10	0.131	2,258,579	390,513	51.5%
11	0.100	2,381,698	368,149	40.5%
12	0.097	2,288,794	335,030	32.8%
13	0.116	2,104,183	299,183	27.6%
14	0.150	1,886,753	262,590	23.3%
15	0.193	1,652,702	225,637	19.3%
16	0.241	1,406,145	188,474	15.5%
17	0.297	1,147,995	151,116	11.6%
18	0.358	878,291	113,586	7.8%
19	0.424	597,079	75,900	4.0%
20	0.500	304,384	38,047	0.0%

597 C Proofs

598 *Proof of 4.1.* In the naive setting, compliance requires $\mathcal{K}(X_1) \leq \mathcal{K}(X_2) \leq \dots \leq \mathcal{K}(X_n)$. The
599 number of such non-decreasing sequences equals the number of multisets of size n from ℓ elements,
600 which is $\binom{n+\ell-1}{n}$ by stars and bars formula [Rosen, 2006]. Dividing by the total ℓ^n sequences gives
601 the result. $\square$

602 *Proof of Theorem 4.1.* Under symmetric label noise,

$$P(Y = 1 \mid Y^* = 1) = 1 - \pi, \quad P(Y = 1 \mid Y^* = 0) = \pi.$$

603 Because $\pi < 1/2$, the posterior $P(Y = 1 \mid Y^*)$ is strictly increasing in Y^* , so any Bayes-optimal
604 ranking is equivalent to scoring by $s = Y^*$. We therefore compute the AUC of the binary oracle
605 score $s \in \{0, 1\}$.

$$AUC = P(s_+ > s_-) + \frac{1}{2}P(s_+ = s_-) \quad (5)$$

606 where s_+ is the score for a random instance with $Y = 1$ and s_- is the score for a random instance
607 with $Y = 0$.

608 The Joint Distribution is:

$$P(Y = 1, Y^* = 1) = (1 - \pi)\rho \quad (6)$$

$$P(Y = 1, Y^* = 0) = \pi(1 - \rho) \quad (7)$$

$$P(Y = 0, Y^* = 1) = \pi\rho \quad (8)$$

$$P(Y = 0, Y^* = 0) = (1 - \pi)(1 - \rho) \quad (9)$$

609 Marginal Distributions:

$$P(Y = 1) = \rho(1 - \pi) + (1 - \rho)\pi \quad (10)$$

$$P(Y = 0) = \rho\pi + (1 - \rho)(1 - \pi) \quad (11)$$

610 Note that $P(Y = 1) + P(Y = 0) = 1$.

611 For the conditional score distributions, define:

$$A = P(s = 1 \mid Y = 1) = \frac{(1 - \pi)\rho}{\rho(1 - \pi) + (1 - \rho)\pi} \quad (12)$$

$$B = P(s = 0 \mid Y = 1) = \frac{\pi(1 - \rho)}{\rho(1 - \pi) + (1 - \rho)\pi} \quad (13)$$

$$C = P(s = 1 \mid Y = 0) = \frac{\pi\rho}{\rho\pi + (1 - \rho)(1 - \pi)} \quad (14)$$

$$D = P(s = 0 \mid Y = 0) = \frac{(1 - \pi)(1 - \rho)}{\rho\pi + (1 - \rho)(1 - \pi)} \quad (15)$$

612 Note that $A + B = 1$ and $C + D = 1$.

613 AUC Calculation:

$$AUC = A \cdot D + \frac{1}{2}(A \cdot C + B \cdot D) \quad (16)$$

$$= A(1 - C) + \frac{1}{2}A \cdot C + \frac{1}{2}(1 - A)(1 - C) \quad (17)$$

$$= A - AC + \frac{1}{2}AC + \frac{1}{2} - \frac{1}{2}A - \frac{1}{2}C + \frac{1}{2}AC \quad (18)$$

$$= \frac{1}{2} + \frac{1}{2}(A - C) \quad (19)$$

614 Simplification for balanced classes:

615 For $\rho = 0.5$:

$$A = \frac{0.5(1 - \pi)}{0.5(1 - \pi) + 0.5\pi} = \frac{0.5(1 - \pi)}{0.5} = 1 - \pi \quad (20)$$

$$C = \frac{0.5\pi}{0.5\pi + 0.5(1 - \pi)} = \frac{0.5\pi}{0.5} = \pi \quad (21)$$

616 Therefore:

$$AUC = \frac{1}{2} + \frac{1}{2}[(1 - \pi) - \pi] = \frac{1}{2} + \frac{1}{2}(1 - 2\pi) = 1 - \pi \quad (22)$$

617 Alternatively, we can use the following expression.

618 Define $X = \rho(1 - \pi) + (1 - \rho)\pi$ and $Y = \rho\pi + (1 - \rho)(1 - \pi)$.

619 The numerator of $(A - C)$ can be computed as:

$$(A - C) \cdot XY = (1 - \pi)\rho \cdot Y - \pi\rho \cdot X \quad (23)$$

$$= \rho[(1 - \pi)Y - \pi X] \quad (24)$$

$$= \rho[(1 - \pi)(\rho\pi + (1 - \rho)(1 - \pi)) - \pi(\rho(1 - \pi) + (1 - \rho)\pi)] \quad (25)$$

$$= \rho[(1 - \pi)^2(1 - \rho) - \pi^2(1 - \rho)] \quad (26)$$

$$= \rho(1 - \rho)[(1 - \pi)^2 - \pi^2] \quad (27)$$

$$= \rho(1 - \rho)(1 - 2\pi) \quad (28)$$

620 Note: $(1 - \pi)^2 - \pi^2 = 1 - 2\pi + \pi^2 - \pi^2 = 1 - 2\pi$.

621 Therefore:

$$A - C = \frac{\rho(1 - \rho)(1 - 2\pi)}{XY} \quad (29)$$

622 and

$$AUC = \frac{1}{2} + \frac{\rho(1 - \rho)(1 - 2\pi)}{2XY} \quad (30)$$

623

□

624 For the thresholded oracle classifier $\hat{Y} = Y^*$, we now compute the resulting F1 score with respect to
625 the observed labels.

626 Since we evaluate performance against the noisy observed labels Y , the confusion matrix elements
627 are:

$$\begin{aligned} TP &= P(\hat{Y} = 1 \cap Y = 1) = P(Y^* = 1 \cap Y = 1) \\ &= P(Y = 1 \mid Y^* = 1) \cdot P(Y^* = 1) = (1 - \pi)\rho \end{aligned} \quad (31)$$

$$\begin{aligned} FP &= P(\hat{Y} = 1 \cap Y = 0) = P(Y^* = 1 \cap Y = 0) \\ &= P(Y = 0 \mid Y^* = 1) \cdot P(Y^* = 1) = \pi\rho \end{aligned} \quad (32)$$

$$\begin{aligned} FN &= P(\hat{Y} = 0 \cap Y = 1) = P(Y^* = 0 \cap Y = 1) \\ &= P(Y = 1 \mid Y^* = 0) \cdot P(Y^* = 0) = \pi(1 - \rho) \end{aligned} \quad (33)$$

$$\begin{aligned} TN &= P(\hat{Y} = 0 \cap Y = 0) = P(Y^* = 0 \cap Y = 0) \\ &= P(Y = 0 \mid Y^* = 0) \cdot P(Y^* = 0) = (1 - \pi)(1 - \rho) \end{aligned} \quad (34)$$

628 For the precision, by definition:

$$\begin{aligned} \text{Precision} &= \frac{TP}{TP + FP} = \frac{(1 - \pi)\rho}{(1 - \pi)\rho + \pi\rho} \\ &= \frac{(1 - \pi)\rho}{\rho[(1 - \pi) + \pi]} = \frac{(1 - \pi)\rho}{\rho} = 1 - \pi \end{aligned} \quad (35)$$

629 **Observation:** Precision is independent of the class distribution ρ and depends only on the noise rate
630 π .

631 For the recall, by definition:

$$\begin{aligned} \text{Recall} &= \frac{TP}{TP + FN} = \frac{(1 - \pi)\rho}{(1 - \pi)\rho + \pi(1 - \rho)} \\ &= \frac{(1 - \pi)\rho}{\rho - \pi\rho + \pi - \pi\rho} = \frac{(1 - \pi)\rho}{\rho(1 - 2\pi) + \pi} \end{aligned} \quad (36)$$

632 **Verification for $\rho = 0.5$:**

$$\text{Recall} = \frac{(1 - \pi) \cdot 0.5}{0.5(1 - 2\pi) + \pi} = \frac{0.5(1 - \pi)}{0.5 - \pi + \pi} = \frac{0.5(1 - \pi)}{0.5} = 1 - \pi \quad (37)$$

633 Thus, for balanced classes, precision = recall = $1 - \pi$.

634 For F1, we use the harmonic mean of precision and recall:

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (38)$$

635 Substituting our expressions:

$$\begin{aligned} F_1 &= \frac{2(1 - \pi) \cdot \frac{(1 - \pi)\rho}{\rho(1 - 2\pi) + \pi}}{(1 - \pi) + \frac{(1 - \pi)\rho}{\rho(1 - 2\pi) + \pi}} \\ &= \frac{2(1 - \pi)^2\rho}{(1 - \pi)[\rho(1 - 2\pi) + \pi] + (1 - \pi)\rho} \end{aligned} \quad (39)$$

636 Simplifying the denominator:

$$\begin{aligned}
& (1 - \pi)[\rho(1 - 2\pi) + \pi] + (1 - \pi)\rho \\
&= (1 - \pi)\rho(1 - 2\pi) + \pi(1 - \pi) + (1 - \pi)\rho \\
&= (1 - \pi)\rho[(1 - 2\pi) + 1] + \pi(1 - \pi) \\
&= (1 - \pi)\rho(2 - 2\pi) + \pi(1 - \pi) \\
&= (1 - \pi)[2\rho(1 - \pi) + \pi]
\end{aligned} \tag{40}$$

637 Therefore:

$$\begin{aligned}
F_1 &= \frac{2(1 - \pi)^2\rho}{(1 - \pi)[2\rho(1 - \pi) + \pi]} = \frac{2(1 - \pi)\rho}{2\rho(1 - \pi) + \pi} \\
&= \frac{2(1 - \pi)\rho}{\pi + 2\rho(1 - \pi)}
\end{aligned} \tag{41}$$

638 **Verification for $\rho = 0.5$:**

$$F_1 = \frac{2(1 - \pi) \cdot 0.5}{\pi + 2 \cdot 0.5 \cdot (1 - \pi)} = \frac{1 - \pi}{\pi + (1 - \pi)} = \frac{1 - \pi}{1} = 1 - \pi \tag{42}$$

639

□

640 C.1 Proof of Theorem 3.3

641 *Proof.* Let $s_{\min} \in \mathcal{S}$ satisfy $\kappa(s_{\min}) = 1$. We first show that the map

$$a \mapsto P(Y^* = 1 \mid X_1 = a)$$

642 is not constant on $\mathcal{A}$.

643 **Case 1:** $\mathcal{S} \subsetneq \mathcal{A}$. Choose $a_0 \in \mathcal{A} \setminus \mathcal{S}$. Because the coordinates of X are independent and have full
644 support, both events $\{X_1 = s_{\min}\}$ and $\{X_1 = a_0\}$ have positive probability, and conditioning on X_1
645 does not change the joint law of $(X_2, \dots, X_n)$.

646 Let $Z_2, \dots, Z_n$ be independent random variables with $Z_t \sim \mathcal{L}(X_t)$, and define

$$\tilde{X} := (s_{\min}, Z_2, \dots, Z_n), \quad \tilde{X}' := (a_0, Z_2, \dots, Z_n).$$

647 Then

$$\tilde{X} \stackrel{d}{=} (X \mid X_1 = s_{\min}), \quad \tilde{X}' \stackrel{d}{=} (X \mid X_1 = a_0).$$

648 Let

$$C := \{\tilde{X} \text{ is compliant}\}, \quad C' := \{\tilde{X}' \text{ is compliant}\}.$$

649 Since $\tilde{X}'_1 = a_0 \notin \mathcal{S}$, any compliant chain in $\tilde{X}'$ must avoid position 1. But $\tilde{X}_t = \tilde{X}'_t$ for every $t \geq 2$,
650 so the same chain is also compliant in $\tilde{X}$. Hence

$$C' \subseteq C.$$

651 Now define the event

$$E := \{Z_{1+\lambda} \in \mathcal{S}\} \cap \bigcap_{t \in \{2, \dots, n\} \setminus \{1+\lambda\}} \{Z_t \notin \mathcal{S}\}.$$

652 Because $n \geq \lambda + 1$, because $\mathcal{S} \subsetneq \mathcal{A}$, and because every symbol has positive probability at every
653 position, we have $P(E) > 0$.

654 On E , the sequence $\tilde{X}$ is compliant via the chain $(1, 1 + \lambda)$: both symbols belong to $\mathcal{S}$, their gap is
655 λ , and

$$\kappa(\tilde{X}_1) = \kappa(s_{\min}) = 1 \leq \kappa(\tilde{X}_{1+\lambda}).$$

656 Hence

$$E \subseteq C.$$

657 On the same event E , the sequence $\tilde{X}'$ is not compliant. Indeed, among positions $2, \dots, n$, the only
 658 key position is $1 + \lambda$, so no compliant chain of length at least 2 can exist. Therefore

$$E \subseteq C \setminus C'.$$

659 Since $C' \subseteq C$, we have

$$P(C) - P(C') = P(C \setminus C').$$

660 Moreover, $E \subseteq C \setminus C'$ and $P(E) > 0$. Therefore

$$P(C) - P(C') \geq P(E) > 0,$$

661 which implies

$$P(C) > P(C').$$

662 Hence

$$P(Y^* = 1 \mid X_1 = s_{\min}) = P(C) > P(C') = P(Y^* = 1 \mid X_1 = a_0).$$

663 **Case 2:** $\mathcal{S} = \mathcal{A}$. Since $n \geq \lambda + 1$, the position $1 + \lambda$ exists. If $X_1 = s_{\min}$, then for every realization
 664 of the rest of the sequence the pair $(1, 1 + \lambda)$ is compliant: both positions lie in $\mathcal{S} = \mathcal{A}$, their gap is
 665 λ , and

$$\kappa(X_1) = \kappa(s_{\min}) = 1 \leq \kappa(X_{1+\lambda}).$$

666 Therefore

$$P(Y^* = 1 \mid X_1 = s_{\min}) = 1.$$

667 Since $\rho = P(Y^* = 1) \in (0, 1)$, we have $\rho < 1$. If

$$P(Y^* = 1 \mid X_1 = a) = 1 \quad \text{for all } a \in \mathcal{A},$$

668 then by the law of total probability,

$$P(Y^* = 1) = \sum_{a \in \mathcal{A}} P(Y^* = 1 \mid X_1 = a) P(X_1 = a) = 1,$$

669 contradicting $\rho < 1$. Hence there exists $a^* \in \mathcal{A}$ such that

$$P(Y^* = 1 \mid X_1 = a^*) < 1.$$

670 Thus

$$P(Y^* = 1 \mid X_1 = s_{\min}) \neq P(Y^* = 1 \mid X_1 = a^*).$$

671 In both cases, there exist $a, b \in \mathcal{A}$ such that

$$P(Y^* = 1 \mid X_1 = a) \neq P(Y^* = 1 \mid X_1 = b).$$

672 Therefore Y^* is not independent of X_1 . Since mutual information between discrete variables is zero
 673 if and only if the variables are independent, it follows that

$$I(Y^*; X_1) > 0.$$

674 Finally, by the symmetric noise model (1),

$$\begin{aligned} P(Y = 1 \mid X_1 = a) &= P(Y = 1 \mid Y^* = 1) P(Y^* = 1 \mid X_1 = a) \\ &\quad + P(Y = 1 \mid Y^* = 0) P(Y^* = 0 \mid X_1 = a) \\ &= (1 - \pi) P(Y^* = 1 \mid X_1 = a) + \pi (1 - P(Y^* = 1 \mid X_1 = a)) \\ &= \pi + (1 - 2\pi) P(Y^* = 1 \mid X_1 = a). \end{aligned}$$

675 Because $\pi < 1/2$, the map

$$u \mapsto \pi + (1 - 2\pi)u$$

676 is strictly increasing. Hence the map

$$a \mapsto P(Y = 1 \mid X_1 = a)$$

677 is also non-constant on $\mathcal{A}$. Thus Y is not independent of X_1 , and again

$$I(Y; X_1) > 0.$$

678

□

679 **C.2 Proof of Corollary ?? with remarks**

680 *Proof.* As in the proof of Theorem 3.3, let $Z_2, \dots, Z_n$ be independent random variables with
 681 $Z_t \sim \text{Uniform}(\mathcal{A})$, and define

$$\tilde{X} = (s_{\min}, Z_2, \dots, Z_n), \quad \tilde{X}' = (a_0, Z_2, \dots, Z_n).$$

682 Then

$$\tilde{X} \stackrel{d}{=} (X \mid X_1 = s_{\min}), \quad \tilde{X}' \stackrel{d}{=} (X \mid X_1 = a_0).$$

683 Let

$$C := \{\tilde{X} \text{ is compliant}\}, \quad C' := \{\tilde{X}' \text{ is compliant}\}.$$

684 As in Theorem 3.3, one has $C' \subseteq C$.

685 Now define the witness event

$$E := \{Z_{1+\lambda} \in \mathcal{S}\} \cap \bigcap_{t \in \{2, \dots, n\} \setminus \{1+\lambda\}} \{Z_t \notin \mathcal{S}\}.$$

686 On E , the sequence $\tilde{X}$ is compliant via the chain $(1, 1 + \lambda)$, while $\tilde{X}'$ is not compliant. Hence

$$E \subseteq C \setminus C'.$$

687 Therefore

$$P(C) - P(C') = P(C \setminus C') \geq P(E).$$

688 By independence and uniformity,

$$P(Z_{1+\lambda} \in \mathcal{S}) = \frac{m}{\ell}, \quad P(Z_t \notin \mathcal{S}) = \frac{\ell - m}{\ell} \quad \text{for each } t \in \{2, \dots, n\} \setminus \{1 + \lambda\}.$$

689 There are exactly $n - 2$ such latter positions, so

$$P(E) = \frac{m}{\ell} \left(\frac{\ell - m}{\ell} \right)^{n-2}.$$

690 Since

$$P(C) = P(Y^* = 1 \mid X_1 = s_{\min}), \quad P(C') = P(Y^* = 1 \mid X_1 = a_0),$$

691 the first claim follows.

692 For the observed label, by (1),

$$P(Y = 1 \mid X_1 = a) = \pi + (1 - 2\pi) P(Y^* = 1 \mid X_1 = a).$$

693 Subtracting the expressions for $a = s_{\min}$ and $a = a_0$ yields

$$P(Y = 1 \mid X_1 = s_{\min}) - P(Y = 1 \mid X_1 = a_0) = (1 - 2\pi) \left(P(Y^* = 1 \mid X_1 = s_{\min}) - P(Y^* = 1 \mid X_1 = a_0) \right),$$

694 and the result follows from the first part. $\square$

695 *Remark C.1* (Scope of the explicit bound). The closed-form expression in Corollary ?? is specific to
 696 the uniform proposal law. Under the more general assumptions of Theorem 3.3, if we write

$$\alpha_t := P(X_t \in \mathcal{S}),$$

697 then the same witness-event argument gives

$$P(Y^* = 1 \mid X_1 = s_{\min}) - P(Y^* = 1 \mid X_1 = a_0) \geq \alpha_{1+\lambda} \prod_{t \in \{2, \dots, n\} \setminus \{1+\lambda\}} (1 - \alpha_t)$$

698 whenever $\mathcal{S} \subsetneq \mathcal{A}$. In the i.i.d. but non-uniform case with common law q , this becomes

$$q(\mathcal{S})(1 - q(\mathcal{S}))^{n-2}.$$

699 No nontrivial positive lower bound depending only on (m, ℓ, n) can hold under mere full support,
 700 since $q(\mathcal{S})$ can be arbitrarily small while remaining positive.

701 *Remark C.2* (Effect of label balancing). Suppose the final experimental sample is obtained from
 702 a proposal law P by retaining examples with probability depending only on the observed label Y .
 703 Equivalently,

$$Q(x, y) = \frac{r_y P(x, y)}{Z}, \quad r_0, r_1 > 0.$$

704 Then

$$Q(X | Y = y) = P(X | Y = y),$$

705 so class-conditional sequence structure, and hence AUC, are unchanged by this reweighting. However,
 706 posterior quantities are transformed. Writing $\beta := r_1/r_0$ and

$$h_\beta(u) := \frac{\beta u}{1 + (\beta - 1)u},$$

707 one has

$$Q(Y = 1 | X_1 = a) = h_\beta(P(Y = 1 | X_1 = a)).$$

708 Since h_β is strictly increasing, label balancing preserves the presence of local information, but
 709 because h_β is nonlinear it can either enlarge or shrink posterior gaps. Consequently, Corollary ??
 710 should be interpreted as a statement about the proposal law, not automatically about the retained
 711 balanced sample.

712 More generally, if

$$\Delta_P := P(Y = 1 | X_1 = s_{\min}) - P(Y = 1 | X_1 = a_0), \quad \Delta_Q := Q(Y = 1 | X_1 = s_{\min}) - Q(Y = 1 | X_1 = a_0),$$

713 then by the mean value theorem,

$$\Delta_Q \geq \min\{\beta, \beta^{-1}\} \Delta_P.$$

714 The same remark applies with Y^* in place of Y if balancing is performed using the latent label rather
 715 than the observed one.

716 C.3 Proof of Theorem 3.5

717 *Proof.* For each $t \in \{1, \dots, n\}$, define

$$B_t := \mathbf{1}\{X_t \in K\}.$$

718 Since $X_t \stackrel{\text{iid}}{\sim} \text{Uniform}(\mathcal{A})$ and

$$p = \frac{|K|}{|\mathcal{A}|},$$

719 we have

$$B_t \stackrel{\text{iid}}{\sim} \text{Bernoulli}(p).$$

720 Let

$$T := \left(\sum_{t=1}^n B_t \right) \bmod 2.$$

721 We first show that Y^* is a fixed bit-flip of T . Using addition modulo 2,

$$\mathbf{1}\{N_k \text{ is even}\} = 1 \oplus (N_k \bmod 2),$$

722 where $\oplus$ denotes XOR. Hence

$$\begin{aligned} Y^* &= \bigoplus_{k \in K} \mathbf{1}\{N_k \text{ is even}\} \\ &= \bigoplus_{k \in K} (1 \oplus (N_k \bmod 2)) \\ &= (|K| \bmod 2) \oplus \bigoplus_{k \in K} (N_k \bmod 2). \end{aligned}$$

723 But

$$\bigoplus_{k \in K} (N_k \bmod 2) = \left(\sum_{k \in K} N_k \right) \bmod 2 = \left(\sum_{t=1}^n \mathbf{1}\{X_t \in K\} \right) \bmod 2 = T.$$

724 Therefore

$$Y^* = (|K| \bmod 2) \oplus T.$$

725 The map $T \mapsto Y^*$ is thus a bijection on $\{0, 1\}$, so for every index set J ,

$$I(Y^*; X_J) = I(T; X_J).$$

726 It is therefore enough to study $I(T; X_J)$.

727 Fix an index set

$$J \subseteq \{1, \dots, n\}, \quad 1 \leq |J| < n.$$

728 Write

$$r := |J|, \quad s := n - r = |J^c| \geq 1.$$

729 Define

$$U_J := \left(\sum_{j \in J} B_j \right) \bmod 2, \quad V_J := \left(\sum_{j \in J^c} B_j \right) \bmod 2.$$

730 Then

$$T = U_J \oplus V_J.$$

731 Since $\{B_j : j \in J\}$ and $\{B_j : j \in J^c\}$ are independent, V_J is independent of X_J . Moreover, U_J is
732 a deterministic function of X_J .

733 Let

$$q_s := P(V_J = 1).$$

734 Because V_J is the parity of s i.i.d. Bernoulli(p) random variables,

$$q_s = \frac{1 - (1 - 2p)^s}{2}.$$

735 Therefore, for every realization x_J of X_J ,

$$P(T = 1 \mid X_J = x_J) = \begin{cases} q_s, & U_J(x_J) = 0, \\ 1 - q_s, & U_J(x_J) = 1. \end{cases}$$

736 **Balanced case:** $p = 1/2$. If $p = 1/2$, then

$$q_s = \frac{1 - (1 - 2p)^s}{2} = \frac{1}{2}.$$

737 Hence, for every realization x_J ,

$$P(T = 1 \mid X_J = x_J) = \frac{1}{2}, \quad P(T = 0 \mid X_J = x_J) = \frac{1}{2}.$$

738 Thus the conditional law of T does not depend on x_J , so T is independent of X_J . Therefore

$$I(T; X_J) = 0,$$

739 and hence

$$I(Y^*; X_J) = 0.$$

740 **Unbalanced case:** $p \neq 1/2$, finite n . Since K is nonempty and proper, $p \in (0, 1)$. If $p \neq 1/2$, then

$$|1 - 2p| < 1 \quad \text{and} \quad (1 - 2p)^s \neq 0$$

741 for every finite $s \geq 1$. Hence

$$q_s \neq \frac{1}{2}.$$

742 Choose any $j_0 \in J$, any $a_{\text{in}} \in K$, and any $a_{\text{out}} \in \mathcal{A} \setminus K$. Define two realizations $x_J, x'_J \in \mathcal{A}^J$ by

$$x_j = a_{\text{out}} \quad \text{for all } j \in J,$$

743 and

$$x'_{j_0} = a_{\text{in}}, \quad x'_j = a_{\text{out}} \quad \text{for all } j \in J \setminus \{j_0\}.$$

744 Then

$$U_J(x_J) = 0, \quad U_J(x'_J) = 1.$$

745 Therefore

$$P(T = 1 \mid X_J = x_J) = q_s, \quad P(T = 1 \mid X_J = x'_J) = 1 - q_s.$$

746 Since $q_s \neq 1/2$, these two conditional probabilities are different. Thus the conditional law of T
747 depends on X_J , so T and X_J are not independent. Hence

$$I(T; X_J) > 0,$$

748 and therefore

$$I(Y^*; X_J) > 0.$$

749 **Asymptotic decay when $p \neq 1/2$.** Now let $J \subset \mathbb{N}$ be any fixed finite index set, and for all
750 sufficiently large n identify it with a subset of $\{1, \dots, n\}$. Then $r = |J|$ is fixed and

$$s = n - r \rightarrow \infty.$$

751 Since $|1 - 2p| < 1$,

$$q_s = \frac{1 - (1 - 2p)^s}{2} \rightarrow \frac{1}{2}.$$

752 Hence, uniformly over all realizations x_J ,

$$P(T = 1 \mid X_J = x_J) \rightarrow \frac{1}{2}.$$

753 Because X_J takes values in the finite set $\mathcal{A}^r$, this implies that the joint law of (T, X_J) converges to
754 the product law of T and X_J . Therefore

$$I(T; X_J) \rightarrow 0.$$

755 Since Y^* is a bijective function of T ,

$$I(Y^*; X_J) = I(T; X_J) \rightarrow 0.$$

756 **Observed noisy label Y .** Assume now that Y is generated from Y^* according to (1) with $\pi < 1/2$.

757 If $p = 1/2$, then Y^* is independent of X_J . Since Y depends on X_J only through Y^* , it follows that
758 Y is also independent of X_J . Hence

$$I(Y; X_J) = 0.$$

759 If $p \neq 1/2$, then

$$\begin{aligned} P(Y = 1 \mid X_J) &= P(Y = 1 \mid Y^* = 1) P(Y^* = 1 \mid X_J) \\ &\quad + P(Y = 1 \mid Y^* = 0) P(Y^* = 0 \mid X_J) \\ &= (1 - \pi) P(Y^* = 1 \mid X_J) + \pi (1 - P(Y^* = 1 \mid X_J)) \\ &= \pi + (1 - 2\pi) P(Y^* = 1 \mid X_J). \end{aligned}$$

760 Because $\pi < 1/2$, the map

$$u \mapsto \pi + (1 - 2\pi)u$$

761 is strictly increasing. Thus whenever $P(Y^* = 1 \mid X_J)$ is non-constant as a function of X_J , so is
762 $P(Y = 1 \mid X_J)$. Therefore, for every finite n ,

$$I(Y; X_J) > 0.$$

763 Finally, since Y is obtained from Y^* through a channel independent of X_J , the data processing
764 inequality gives

$$I(Y; X_J) \leq I(Y^*; X_J).$$

765 Hence, for every fixed finite $J \subset \mathbb{N}$,

$$I(Y; X_J) \rightarrow 0 \quad \text{as } n \rightarrow \infty.$$

766 This proves all claims. In particular, when $p = 1/2$, we have

$$I(Y^*; X_J) = 0 \quad \text{for every } J \subseteq \{1, \dots, n\} \text{ with } |J| < n.$$

767 Therefore the task is k -locally pure for every $k < n$. □

D Models and Hyperparameter Configurations

This appendix provides complete hyperparameter configurations for all models evaluated in our experiments.

D.1 Training Procedure

Optimization. All neural models are trained using the AdamW optimizer [Loshchilov and Hutter, 2019] with weight decay. For pretrained LLMs and BERT, we use a linear learning rate schedule with 6% warmup steps. We employ cross-entropy loss for binary classification. Training uses early stopping based on validation loss with patience of 3 epochs for pretrained models and 5 epochs for models trained with randomly initialized parameters.

Pretrained models. LLMs (Llama, Qwen) and BERT are fine-tuned with learning rate 2×10^{-5} , batch size 16, and maximum sequence length 200 tokens. To enable efficient fine-tuning, we apply LoRA [Hu et al., 2021] with rank $r = 8$, $\alpha = 16$, and dropout 0.1, targeting the attention projection matrices (q_proj, k_proj, v_proj, o_proj for LLMs; query, key, value for BERT). Large models employ gradient checkpointing and mixed-precision training (bfloat16) to fit within GPU memory constraints. We train for a maximum of 20 epochs with patience of 3 epochs.

Randomly initialized Models. LSTM, BiLSTM, Transformer, and hybrid architectures are trained with batch size 64 for up to 30 epochs. Learning rates are tuned per architecture: 10^{-3} for recurrent models (LSTM, BiLSTM), 5×10^{-4} for Transformer-based models, and 10^{-4} for the RNN-Transformer. Dropout rates range from 0.2 to 0.3 depending on model capacity. Early stopping monitors validation F1 score with patience of 5 epochs.

XGBoost baselines. XGBoost models use learning rate 0.05, maximum depth 6, subsample ratio 0.8, and column sampling ratio 0.8 per tree. We apply L1 regularization ($\alpha = 0.5$) and L2 regularization ($\lambda = 1.5$) with minimum child weight 2 and minimum split gain $\gamma = 0.1$. Class imbalance is addressed via `scale_pos_weight`. Training runs for up to 200 boosting rounds with early stopping (patience 10) based on validation log-loss.

Threshold optimization. For all models, we optimize the classification threshold on the validation set by searching over $\{0.10, 0.11, \dots, 0.89\}$ to maximize F1 score, rather than using the default 0.5 threshold.

Table 7: Models tested and relative dimension. For XGBoost, we report hyperparameter configuration in Table 14. As tree ensemble complexity is not directly comparable to neural network parameter counts, we omit this metric; the trained model typically comprised 80–120 trees with maximum depth 6.

Category	Model	Parameters
Pretrained	BERT-base	110M
	RoBERTa	355M
	Llama-3.2-1B	1.0B
	Qwen3-4B-Think	4.0B
	Llama-3.1-8B	8.0B
	Qwen2.5-14B	14B
	Llama-3.1-70B	70B
Randomly initialized	Llama-1M	1M
	LSTM	198K–340K
	BiLSTM	256K–395K
	Transformer	103K–104K
	RNN-Transformer*	319K–455K
Tree-based	XGBoost	–

*We call RNN-Transformer a BiLSTM-Transformer.

796 D.2 Pretrained Language Models

797 Table 8 summarizes the fine-tuning configuration for pretrained LLMs.

Table 8: Fine-tuning configuration for pretrained LLMs (Llama, Qwen families).

Hyperparameter	Value
<i>Training</i>	
Learning rate	2×10^{-5}
Optimizer	AdamW
Batch size	16
Max epochs	20
Early stopping patience	3
Early stopping criterion	Validation loss
Warmup ratio	6% of total steps
Scheduler	Linear decay
<i>LoRA Configuration</i>	
Rank (r)	8
Alpha (α)	16
Dropout	0.1
Target modules	q_proj, k_proj, v_proj, o_proj
Bias	None
Task type	CAUSAL_LM
<i>Input Processing</i>	
Max sequence length	512 tokens
Padding	Max length
Truncation	True
<i>Memory Optimization</i>	
Precision	bfloat16
Gradient checkpointing	Enabled
Use cache	Disabled

798 Table 9 shows the architectural configurations for small-scale Llama models trained with randomly
799 initialized parameters.

Table 9: Architecture configurations for Llama models trained with randomly initialized parameters.

Model	Hidden size	Layers	Heads	KV Heads	Intermediate size
Llama-1M	128	4	4	2	512

800 D.3 BERT Models

801 Table 10 summarizes the fine-tuning configuration for BERT-based models.

802 D.4 Recurrent and Transformer Architectures (Randomly Initialized)

803 Table 11 summarizes the training configuration shared across all models trained from scratch.

804 Table 12 and Table 13 present the default and optimized architecture configurations, respectively.

805 Additional architecture details:

- 806 • **Transformer:** 4 attention heads; GELU activation; learnable positional encodings; mean
807 pooling with padding mask.
- 808 • **LSTM:** Final hidden state used for classification.
- 809 • **RNN-Transformer:** BiLSTM output projected to embedding dimension, then processed by
810 Transformer encoder.

Table 10: Fine-tuning configuration for BERT models.

Hyperparameter	Value
<i>Training</i>	
Learning rate	2×10^{-5}
Optimizer	AdamW
Batch size	16
Max epochs	20
Early stopping patience	3
Early stopping criterion	Validation loss
Warmup ratio	6% of total steps
Scheduler	Linear decay
<i>LoRA Configuration (when used)</i>	
Rank (r)	8
Alpha (α)	16
Dropout	0.1
Target modules	query, key, value
Bias	None
Task type	SEQ_CLS
<i>Input Processing</i>	
Max sequence length	512 tokens
Padding	Max length
Truncation	True

Table 11: Training configuration for models trained with randomly initialized parameters.

Hyperparameter	Value
Optimizer	Adam
Batch size	64
Max epochs	30
Early stopping patience	5
Early stopping criterion	Validation F1
Loss function	Cross-entropy

811 D.5 XGBoost Baselines

812 Table 14 presents the XGBoost configuration.

813 For XGBoost-LLM, embeddings are extracted from Llama-3.2-1B using the prompt format described
814 in Section 4.4, with mean pooling over the last hidden layer (dimension 2048).

815 D.6 Data Fractions

816 To assess data efficiency, we train each model on the following fractions of the training set:
817 {1%, 10%, 30%, 50%, 75%, 100%}. Subsampling is performed using stratified sampling to maintain
818 class balance across all fractions.

819 **Reproducibility.** All experiments use a fixed random seed (9550) for data splitting, weight initial-
820 ization, and stochastic optimization. Each configuration is run with three different seeds; we report
821 means and standard deviations (see 3).

822 D.7 Hardware and Software

823 Experiments are conducted on a single NVIDIA A100 GPU (80GB). Fine-tuning Llama-8B requires
824 approximately 128GB CPU memory with gradient checkpointing enabled. Training times range from
825 minutes (small models, 1% data) to several hours (Llama-8B, full data).

Table 12: Default architecture configurations for randomly initialized models.

Model	Embed. dim	Hidden dim	Layers	Dropout	Learning rate
Transformer	64	256 (FFN)	2	0.3	5×10^{-4}
LSTM	32	64	2	0.3	10^{-3}
RNN-Transformer*	64	64 (RNN) / 256 (FFN)	1+2	0.3	5×10^{-4}

*We call RNN-Transformer a BiLSTM-Transformer.

Table 13: Optimized architecture configurations for models randomly initialized.

Model	Embed. dim	Hidden dim	Layers	Dropout	Learning rate
Transformer	64	128 (FFN)	3	0.3	5×10^{-4}
LSTM	32	128	2	0.2	2×10^{-3}
BiLSTM	64	64	3	0.2	10^{-3}
RNN-Transformer*	128	32 (RNN) / 256 (FFN)	1+3	0.3	10^{-4}

*We call RNN-Transformer a BiLSTM-Transformer.

826 E Results

Table 14: XGBoost hyperparameter configuration.

Hyperparameter	Value
<i>Tree Parameters</i>	
Max depth	6
Min child weight	1
Gamma (min split loss)	0.1
Subsample	0.8
Column sample by tree	0.8
<i>Regularization</i>	
L1 regularization (α)	0.5
L2 regularization (λ)	1.5
<i>Training</i>	
Learning rate (η)	0.05
Max boosting rounds	200
Early stopping patience	10
Early stopping criterion	Validation log-loss
Tree method	hist
<i>Class Imbalance</i>	
Scale pos weight	$N_{\text{negative}}/N_{\text{positive}}$

Model Comparison on Naive Dataset

Test metrics by training set size (shaded: estimated variability across runs)

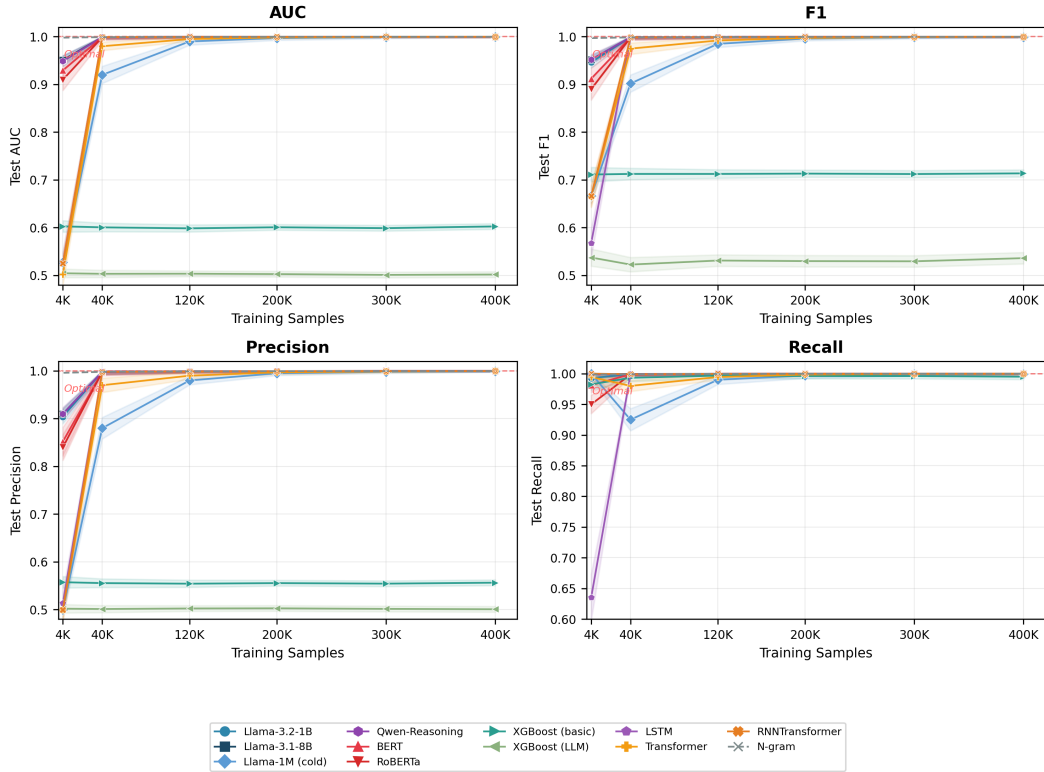


Figure 6: Model performance on the Naive dataset across four metrics. All models achieve near-perfect performance, with the exception of XGBoost. Shaded regions indicate variability across runs.

Table 15: Full classification performance across models and datasets (100% training data). Standard deviations in parentheses.

Model	AUC	F1	Prec	Rec
<i>Naive ($\pi = 0.0$)</i>				
<i>Optimal</i>	1.000	1.000	1.000	1.000
N-gram	.999	.999	.999	.999
XGBoost (basic)	.603 (.004)	.714 (.005)	.556 (.004)	.995 (.003)
XGBoost (LLM)	.502 (.004)	.536 (.008)	.501 (.004)	.577 (.010)
BERT	.999 (.001)	.999 (.001)	.999 (.001)	.999 (.001)
RoBERTa	.999 (.001)	.999 (.001)	.999 (.001)	.999 (.001)
Llama-3.2-1B	.999 (.001)	.999 (.001)	.999 (.001)	.999 (.001)
Llama-3.1-8B	.999 (.001)	.999 (.001)	.999 (.001)	.999 (.001)
Llama-1M (cold)	.999 (.001)	.999 (.001)	.999 (.001)	.999 (.001)
Qwen3-4B-Think	.999 (.001)	.999 (.001)	.999 (.001)	.999 (.001)
LSTM	.999 (.001)	.999 (.001)	.999 (.001)	.999 (.001)
Transformer	.999 (.001)	.999 (.001)	.999 (.001)	.999 (.001)
RNN-Transformer [†]	.999 (.001)	.999 (.001)	.999 (.001)	.999 (.001)
<i>Tricky Deterministic ($\pi = 0.0$)</i>				
<i>Optimal</i>	1.000	1.000	1.000	1.000
2-gram	.821	.672	.593	.775
XGBoost (basic)	.625 (.012)	.457 (.015)	.516 (.010)	.409 (.018)
XGBoost (LLM)	.494 (.010)	.399 (.012)	.353 (.008)	.459 (.015)
BERT	.999 (.001)	.999 (.001)	.999 (.001)	1.000 (.001)
RoBERTa	.999 (.001)	.999 (.001)	.999 (.001)	1.000 (.001)
Llama-3.2-1B	.846 (.015)	.695 (.018)	.623 (.020)	.785 (.022)
Llama-3.1-8B	.995 (.003)	.984 (.008)	.976 (.010)	.993 (.007)
Llama-1M (cold)	.832 (.012)	.686 (.015)	.605 (.018)	.791 (.020)
Qwen3-4B-Think	.850 (.014)	.701 (.016)	.620 (.017)	.800 (.020)
LSTM	.999 (.001)	.994 (.003)	.995 (.003)	.994 (.003)
Transformer	.999 (.001)	.992 (.002)	.985 (.005)	1.000 (.001)
RNN-Transformer [†]	.997 (.002)	.992 (.003)	.985 (.005)	1.000 (.002)
<i>Tricky Random ($\pi = 0.3$)</i>				
<i>Optimal</i>	.670	.577	.700	.491
2-gram	.585	.592	.420	1.000
XGBoost (basic)	.463 (.010)	.418 (.012)	.437 (.008)	.400 (.015)
XGBoost (LLM)	.418 (.011)	.453 (.014)	.437 (.008)	.470 (.018)
BERT	.669 (.006)	.592 (.004)	.420 (.005)	1.000 (.002)
RoBERTa	.652 (.008)	.592 (.003)	.420 (.005)	1.000 (.002)
Llama-3.2-1B	.587 (.008)	.592 (.005)	.420 (.006)	.999 (.005)
Llama-3.1-8B	.575 (.008)	.591 (.005)	.425 (.006)	.968 (.010)
Qwen3-4B-Think	.597 (.007)	.591 (.004)	.423 (.005)	.979 (.008)
Qwen2.5-14B*	.670 (—)	.586 (—)	.476 (—)	.763 (—)
LSTM	.665 (.003)	.562 (.020)	.660 (.002)	.489 (.031)
Transformer	.671 (.001)	.579 (.003)	.701 (.015)	.492 (.012)
RNN-Transformer [†]	.671 (.005)	.578 (.005)	.683 (.010)	.501 (.005)
<i>Parity Deterministic ($\pi = 0.0$)</i>				
<i>Optimal</i>	1.000	1.000	1.000	1.000
1-gram	.505	.668	.501	1.000
XGBoost (basic)	.495 (.004)	.490 (.005)	.497 (.003)	.484 (.006)
XGBoost (LLM)	.501 (.003)	.639 (.005)	.502 (.002)	.881 (.008)
BERT	.503 (.002)	.668 (.002)	.501 (.002)	1.000 (.001)
RoBERTa	.501 (.003)	.668 (.002)	.501 (.002)	1.000 (.001)
Llama-3.2-1B	.497 (.005)	.668 (.001)	.501 (.002)	1.000 (.001)
Llama-3.1-8B	.500 (.003)	.668 (.002)	.501 (.002)	1.000 (.001)
Llama-1M (cold)	.503 (.003)	.668 (.002)	.501 (.003)	1.000 (.001)
Llama-3.3-70B*	.502 (—)	.668 (—)	.501 (—)	1.000 (—)
Qwen3-4B-Think	.500 (.002)	.668 (.001)	.501 (.000)	1.000 (.001)
LSTM	.502 (.003)	.668 (.002)	.501 (.002)	1.000 (.001)
Transformer	.501 (.003)	.668 (.002)	.501 (.002)	1.000 (.001)
RNN-Transformer [†]	.501 (.002)	.668 (.002)	.501 (.003)	1.000 (.001)

*just one run for time constraint, no standard deviation.

[†]We call RNN-Transformer a BiLSTM-Transformer.

Model Comparison on Tricky Deterministic Dataset

Test metrics by training set size (shaded: estimated variability across runs)

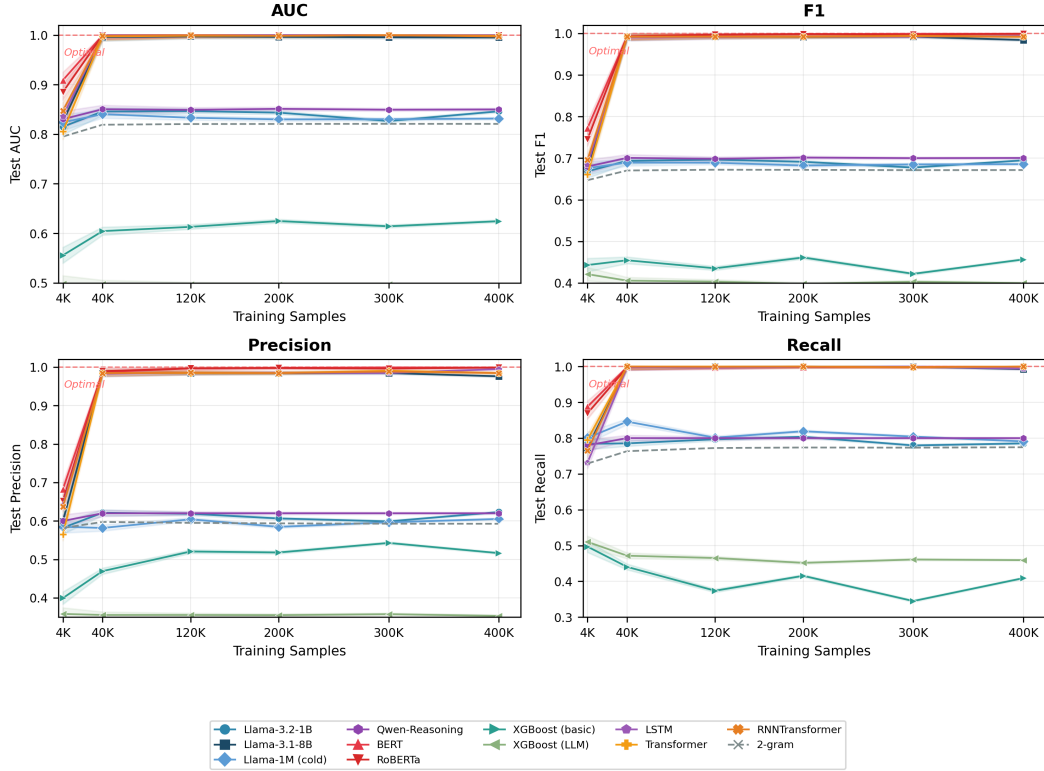


Figure 7: Model performance on the Tricky Deterministic dataset across four metrics. Despite the deterministic label function ($\pi = 0.0$), a clear separation emerges between model families. Bidirectional encoders (BERT, RoBERTa), recurrent architectures (LSTM, RNN-Transformer), and larger causal decoders (Llama-8B) achieve near-perfect performance, demonstrating their ability to capture long-range dependencies when the relevant token can appear anywhere in the sequence. In contrast, smaller causal decoders (Llama-1B, Qwen-7B) plateau at ~ 0.85 AUC, suggesting that autoregressive models require sufficient scale to compensate for the causal attention mask’s limitation on integrating future context. Notably, the Llama-1M cold start model, trained from randomly-initialized weights, exhibits the same plateau, indicating that this capacity threshold is intrinsic to the architecture rather than an artifact of pretraining. The N-gram baseline saturates at ~ 0.82 , confirming that local patterns alone are insufficient. Shaded regions indicate variability across runs.

Model Comparison on Tricky Random Dataset

Test metrics by training set size (shaded: estimated variability across runs)

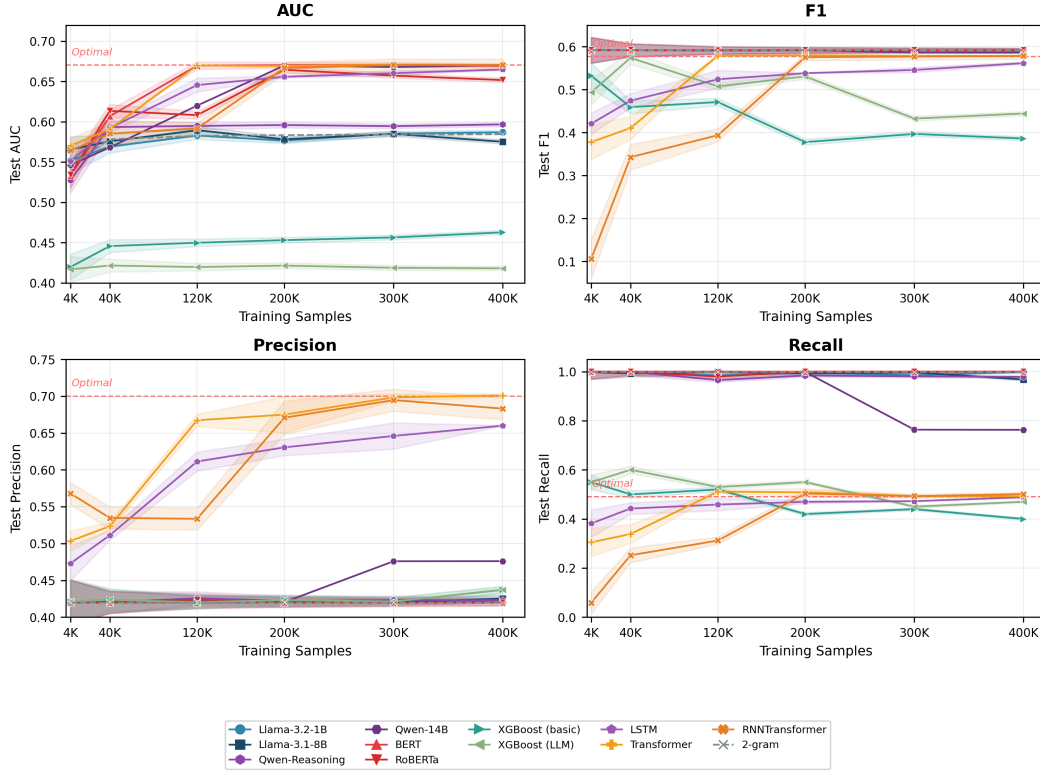


Figure 8: Model performance on the Tricky Random dataset across four metrics. The introduction of stochastic labels ($\pi = 0.3$) establishes an optimal AUC of 0.67, providing an upper bound against which all models can be evaluated. Bidirectional encoders (BERT, RoBERTa), recurrent architectures (LSTM, Transformer, RNN-Transformer), and larger causal decoders (Qwen-14B) approach this theoretical limit, confirming their capacity to extract the maximum learnable signal from the sequence. Smaller causal decoders (Llama-1B, Llama-8B, Qwen-4B) plateau at ~ 0.58 – 0.60 , near the N-gram baseline, suggesting that decoder-only models require substantially more parameters to overcome the autoregressive constraint under label noise. Interestingly, both XGBoost variants perform below chance (< 0.50), indicating that their feature representations actively mislead the classifier, basic n-gram features fail to capture the relevant long-range dependency, while LLM embeddings appear to encode spurious correlations. This dataset highlights how label noise amplifies the parameter requirements for causal architectures to match bidirectional models. Shaded regions indicate variability across runs.

Model Comparison on Parity Deterministic Dataset

Test metrics by training set size (shaded: estimated variability across runs)

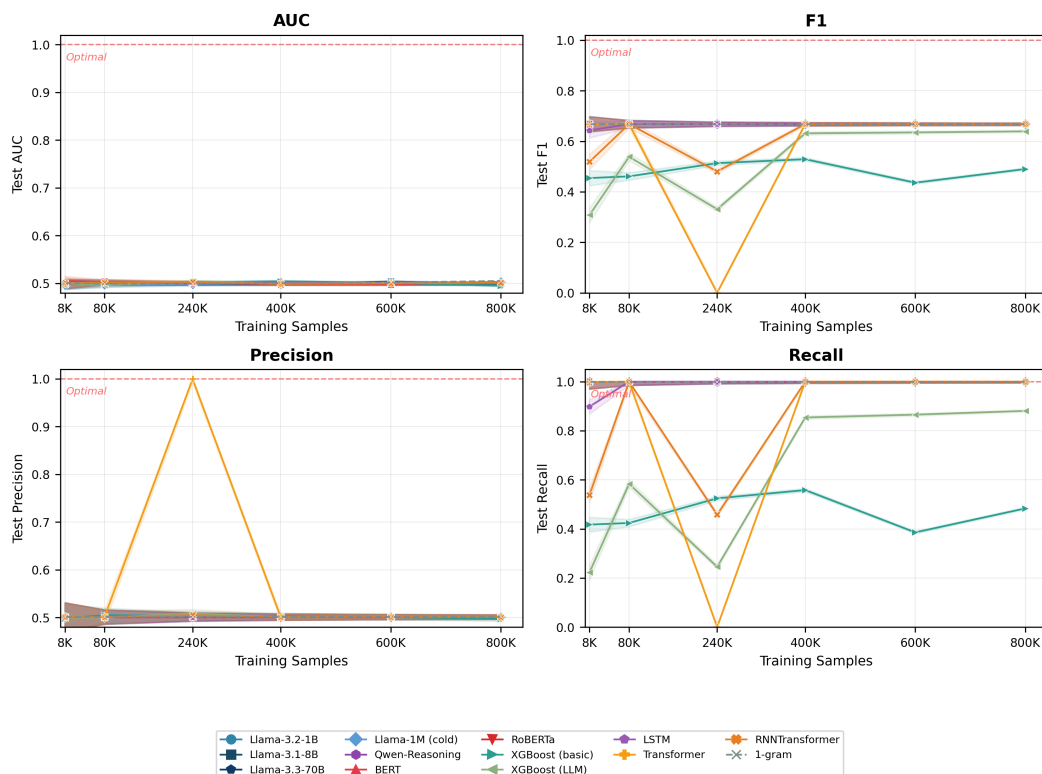


Figure 9: Model performance on the Parity dataset across four metrics. Despite the deterministic label function ($\pi = 0.0$), all models remain at chance level (~ 0.50 AUC), revealing a fundamental limitation shared across architectures. The parity task requires counting the occurrences of a target token modulo 2; a computation that demands precise tracking of state across the entire sequence. Neither bidirectional encoders (BERT, RoBERTa), recurrent architectures (LSTM, RNN-Transformer), nor causal decoders (Llama, Qwen) show any learning signal, suggesting that current neural architectures struggle with tasks requiring exact counting or discrete state maintenance over long contexts. This result aligns with theoretical work on the expressiveness limitations of transformers for inherently sequential computations. The N-gram baseline also fails, as expected, since parity is a global property that cannot be inferred from local patterns. This task thus serves as a challenging benchmark that remains unsolved by standard fine-tuning approaches, pointing toward the need for architectural innovations or alternative training strategies to handle such computations. Shaded regions indicate variability across runs.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer **[N/A]** means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer **[N/A]** means that the paper does not include experiments.
- If the paper includes experiments, a **[No]** answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.

- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.

- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.

- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

1134 Question: Does the paper describe the usage of LLMs if it is an important, original, or
1135 non-standard component of the core methods in this research? Note that if the LLM is used
1136 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
1137 scientific rigor, or originality of the research, declaration is not required.

1138 Answer: **[TODO]**

1139 Justification: **[TODO]**

1140 Guidelines:

- 1141 • The answer [N/A] means that the core method development in this research does not
1142 involve LLMs as any important, original, or non-standard components.
- 1143 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
1144 be described.